Retro 2

CBOT

Uraib Shahid
Haosen Wang
Carolina
Francisco Pimentel
Nicolo Marras
Vladyslav Pernarovkyi
Parniyan Ahmadi

Team Members



Nicòlo Marras
Blogger



Parniyan Ahmadi (Aria)
Developer



Vladyslav Pernarovskyi
Team Leader



Uraib Shahid
Developer



Francisco Pimentel
Developer



Haosen Wang (Sunny)
Developer

Table of Contents

Team Members *2*

Table of Contents *3*

Index of Images *6*

Index of Tables *7*

Introduction *8*

Forword **8**

Project Objective **9**

Maze Scenarios **9**

Requirements Analysis *10*

MSCW Method **10**

System Architecture *11*

Overall System Architecture **11**

Robot Boards: BB011, BB016, BB046 ..... **12**

BB011 13

BB016 14

BB046 15

Separation Between Hardware, Software, and Behavior **16**

Hardware Modules (/common/robot/) 16

Reusable Utilities (/common/tools/ and /common/constant/) 16

Behavior Programs (/maps_program/) 17

Entry Point (main.cpp) 17

Communication and Monitoring Architecture **18**

Architectural Benefits **18**

User Stories *19*

Hardware Design *20*

Motors and Pulse Sensors (LM393) **20**

Ultrasonic Sensor - HC-SR04 ..... **23**

Reflective Sensor Array (8-Channel Infrared) **25**

Gripper Servo - SG90 ..... **28**

Neopixel LEDs **30**

HC-12 Wireless Communication Module **33**

Summary of Hardware Contribution **35**

Software Architecture *36*

Development Environment **36**

Project Folder Structure **36**

/common/ 36

Behavior Programs (/maps_program/) **38**

Entry Point - /main.cpp **38**

Board Configuration Using Preprocessor Directives **39**

Navigation Logic ***39***

Line Following **39**

Sensor Patter Interpretation 39

Motor Correction Strategy 40

Line Maze **40**

Intersection Detection 40

Timed Turning Logic 40

Physical Maze **41**

Obstacle Detection 41

Decision-Making Strategy 41

Movement Execution 41

Non-Blocking Execution and Stability **42**

Communication System ***42***

Network Diagram **42**

Communication Overview **43**

Robot-Side Communication (HC-12) **43**

Data Types Transmitted 43

Transmission Control 43

WEB **44**

Base Station Architecture 44

Python Backend System 44

Frontend Dashboard 44

User Feedback and Debugging 45

Database and Data Model **45**

Data Storage 45

Entity-Relationship Diagram (ERD) 46

Testing and Measurements ***47***

Testing Strategy **47**

Advisory Report **47**

Identified Issues 47

Implemented Improvements 47

Measurement Report **48**

Performance Metrics 48

Results and Analysis 48

Integrated System Testing **48**

Version History *52*

Weekly Reflections *53*

Reflection Method **53**

Weekly Reflection Summary **53**

Weekly Reflections (What-How-What) **54**

Week 1 54
Week 2 54
Week 3 54
Week 4 55
Week 5 55
Week 6 55
Week 7 56
Week 8 56

Code *57*

Project Layout **57**

Main Application File **57**

main.cpp 57

Common Modules **59**

/common/constant/ 59
/common/robot/ 63
/common/tools/ 83

Behaviour Programs **94**

/maps_program/follow_single_line/ 94
/maps_program/maze_line/ 96
/maps_program/physical_maze/ 99

Debriefing *103*

Additional Links *103*

Index of Images

Figure 1 - BB011 | Schematic View 1 13
Figure 2 - BB011 | Schematic View 2 13
Figure 3 - BB016 | Schematic View 1 14
Figure 4 - BB016 - Schematic View 2 14
Figure 5 - BB046 | Schematic View 1 15
Figure 6 - BB046 | Schematic View 2 15
Figure 7 - Motors 20
Figure 8 - Sonar 23
Figure 9 - Reflective Sensor 25
Figure 10 - Gripper 28
Figure 11 - Neopixel 30
Figure 12 - HC12 33
Figure 13 - Network Diagram 42
Figure 14 - Webpage 45
Figure - Entity-Relationship Diagram (ERD) 46

Index of Tables

Table 1 - MSCW Method 10
Table 2 - Motors | Specification 21
Table 3 - Sonar | Specification 24
Table 4 - Reflective Sensor | Specification 26
Table 5 - Gripper | Specification 28
Table 6 - Neopixel | Specification 30
Table 7 - HC12 | Specification 34
Table 8 - Summary of Hardware Contribution 35
Table 9 - Reflective Sensor | BB016 49
Table 10 - Sonar - BB016 50
Table 11 - Neopixel | BB016 51
Table 12 - Version History 53
Table 13 - Week 1 Reflection 54
Table 14 - Week 2 Reflection 54
Table 15 - Week 3 Reflection 55
Table 16 - Week 4 Reflection 55
Table 17 - Week 5 Reflection 55
Table 18 - Week 6 Reflection 56
Table 19 - Week 7 Reflection 56
Table 20 - Week 8 Reflection 56

Introduction

Forword

This portfolio documents the complete engineering process behind the development of the Retro2CBot team, an Arduino-based autonomous robot designed to navigate and solve multiple competitive maze environments.

The project was developed with a strong focus on modularity, scalability, and clear separation between hardware control and behavioral logic. Rather than building a robot for a single task, the team designed a reusable robotic architecture capable of solving 3 distinct scenarios: line following, line maze, and physical maze.

Throughout the development process, a special attention was given to writing reusable code modules, isolating low-level hardware interaction from high-level navigation strategies, and ensuring that the same software base could be deployed across different robot boards with minimal changes.

To guide the project, the team applied the MSCW prioritization method for requirement management and the What-How-What reflection method for continuous technical and collaborative improvement.

This document presents not only the final robot, but also the architectural decisions, design strategies, testing procedures, and engineering reasoning that led to its successful implementation.

Project Objective

The main objective of the team project was to design and implement an autonomous robot capable of efficiently navigating and completing 3 different maze challenges:

1. A line following track, where the robot must continuously detect and follow a path using reflective sensors.
2. A line maze requires the robots to make time turning decisions at intersections.
3. A physical maze, where the robot must detect walls and apply decision-making rules to find a path without prior knowledge of the environment.

The robot was required to operate fully autonomously, minimize completion time, and transmit real time operational data to an external monitoring system through wireless communication.

Beyond solving the mazes, the project aimed to develop robust robotic software architecture where sensors, motors, communication modules and navigation logic are cleanly separated into reusable components. This allows the same system to be adapted to different robot configurations and Mace types without rewriting the core code base.

Maze Scenarios

The team was specifically designed to handle 3 distinct navigation scenarios, each requiring a different sensing strategy and control logic.

Line following relies on the reflective sensor array to continuously detect the position of a black line and adjust motor speeds to remain centered on the track.

Line maze extends this behavior by introducing intersections where the robot must execute precise time turns based on sensor patterns and internal timing logic.

The physical maze removes line guidance entirely and requires the robots to rely on ultrasonic sensors to detect walls and apply a decision-making strategy to navigate to the environment.

Requirements Analysis

MSCW Method

Priority	Requirement	Notes
Must Have	Basic movement, object avoidance, HC-12 connectivity and database	These are the core robot functionalities; they are required for the robot to navigate safely and to complete the assigned tasks.
Should Have	Real-time wireless data logging	Real-time wireless data logging to transmit robot status (e.g., LEDs, sonar, reflective sensor, gripper) using HC-12 communication and WebSocket technology for live monitoring.
Could Have	3 mazes in one	This code could solve 3 mazes using only one robot, with small changes required. Not needed for the assessment
Won't Have	Multi-robot coordination	Not needed for the assessment

Table 1 - MSCW Method

System Architecture

Overall System Architecture

The team was designed following a clear architectural principle: separation between hardware control, reusable software modules, and high-level navigation behavior.

Instead of writing a single monolithic program to solve a maze, the system was divided into independent layers that interact with each other:

- Low-level hardware layer: responsible for direct interaction with motors, sensors, LEDs, gripper and communication modules;
- Reusable software layer contains common tools, constants, and helper modules that can be shared across different robots and programs;
- Behavior layer: implements the navigation logic required to solve each type of maze by combining low-level components;
- Communication and monitoring layer transmit robots' data wirelessly to an external dashboard and database for real-time monitoring.

This separation allows the same code base to be reused across multiple robot boards and may scenarios with minimal changes, ensuring modularity and scalability.

Robot Boards: BB011, BB016, BB046

During the project, three different robot boards were used: BB011, BB016, and BB046. Each Board has different pin configurations and hardware layouts.

Instead of rewarding rewriting code for each robot, a configuration approach based on preprocessor directives (define) was implemented in board.h. By selecting the desired board before compilation, the entire project automatically adapts:

- Motor pins;
- Sensor pins;
- Neopixel pin;
- HC-12 communication pins;
- Gripper pin;
- PID parameters.

This design ensures that the robot software is hardware-independent, and the same logic can run on different physical robots.

BB011

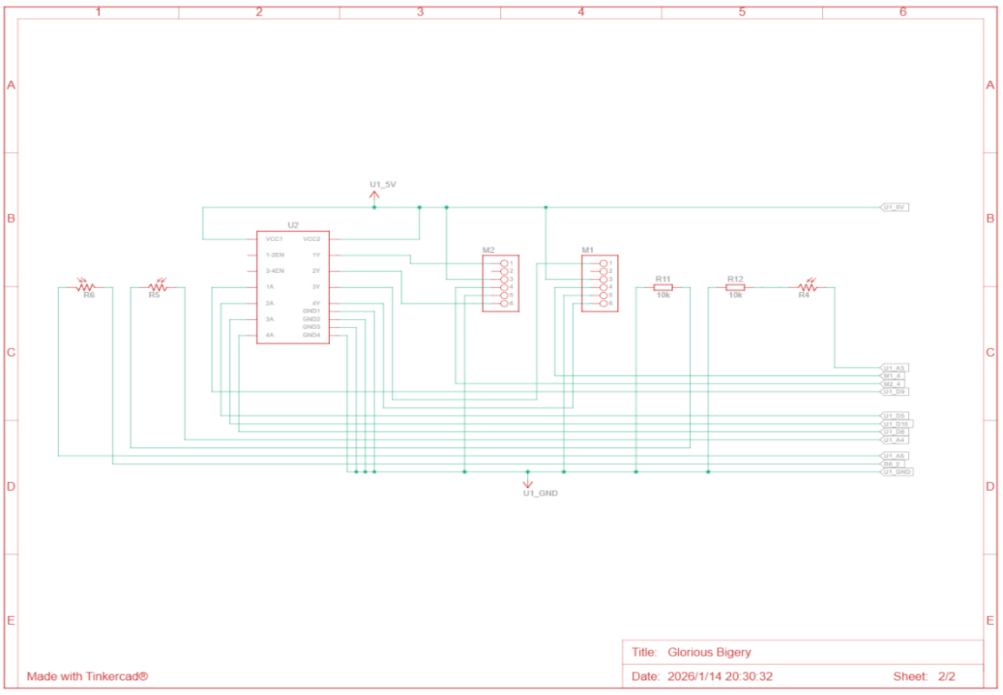


Figure 1 - BB011 | Schematic View 1

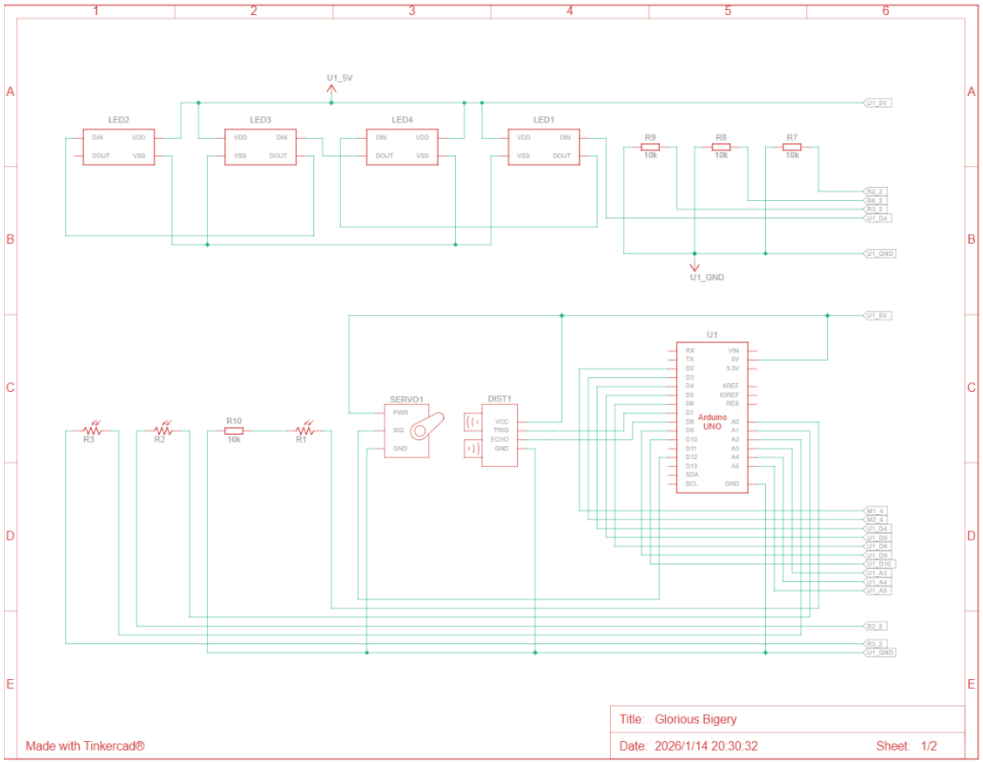


Figure 2 - BB011 | Schematic View 2

BB016

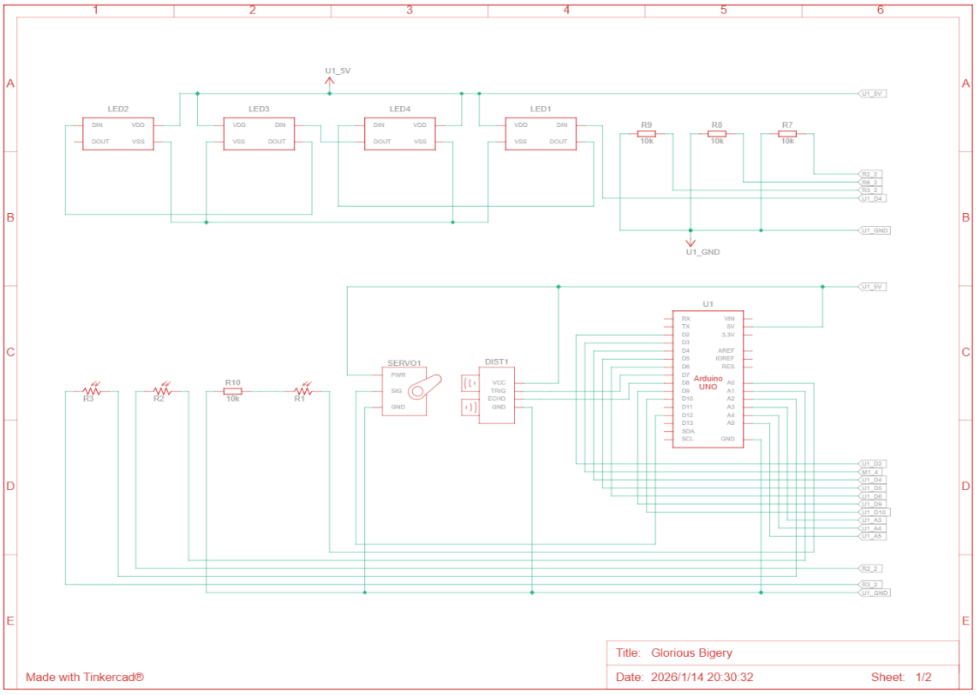


Figure 3 - BB016 | Schematic View 1

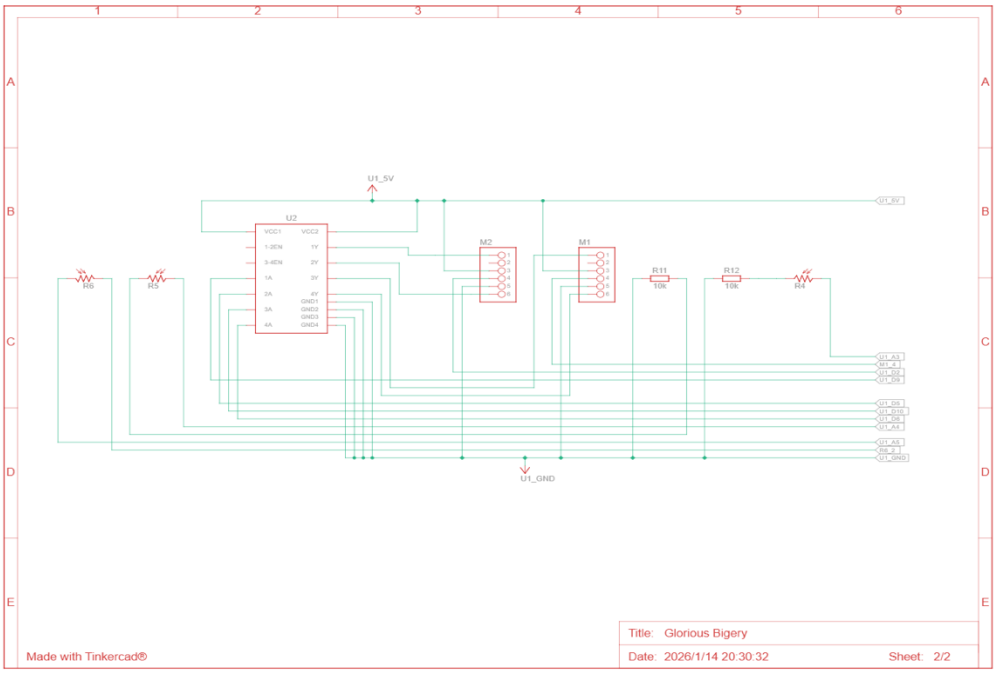


Figure 4 - BB016 - Schematic View 2

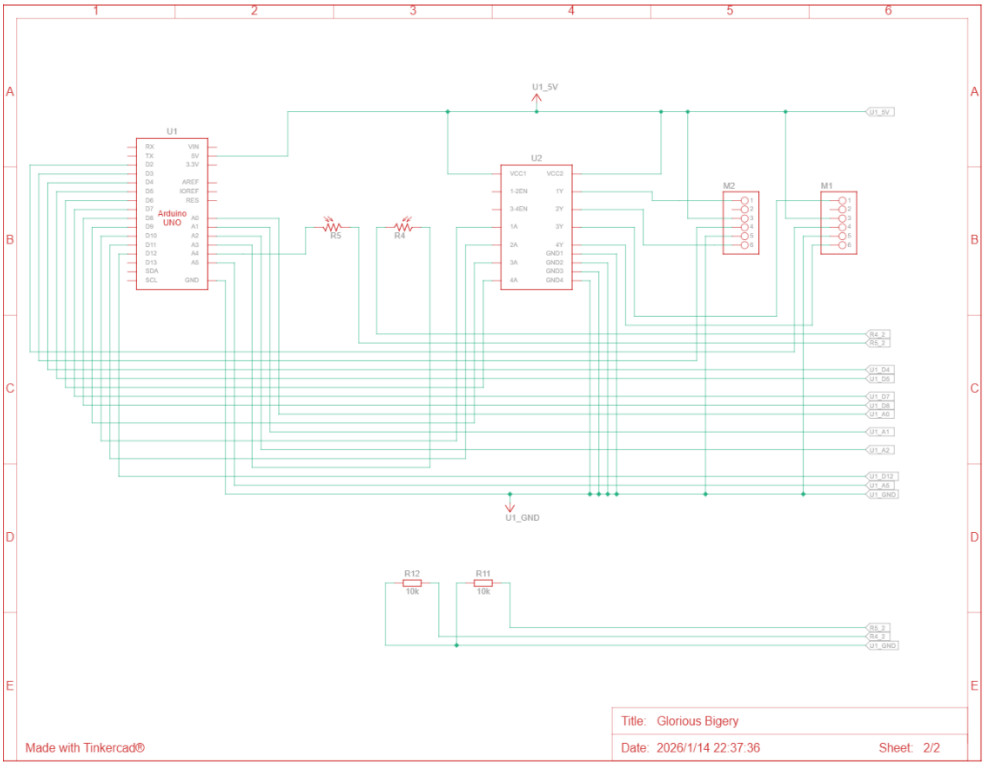


Figure 5 - BB046 | Schematic View 1

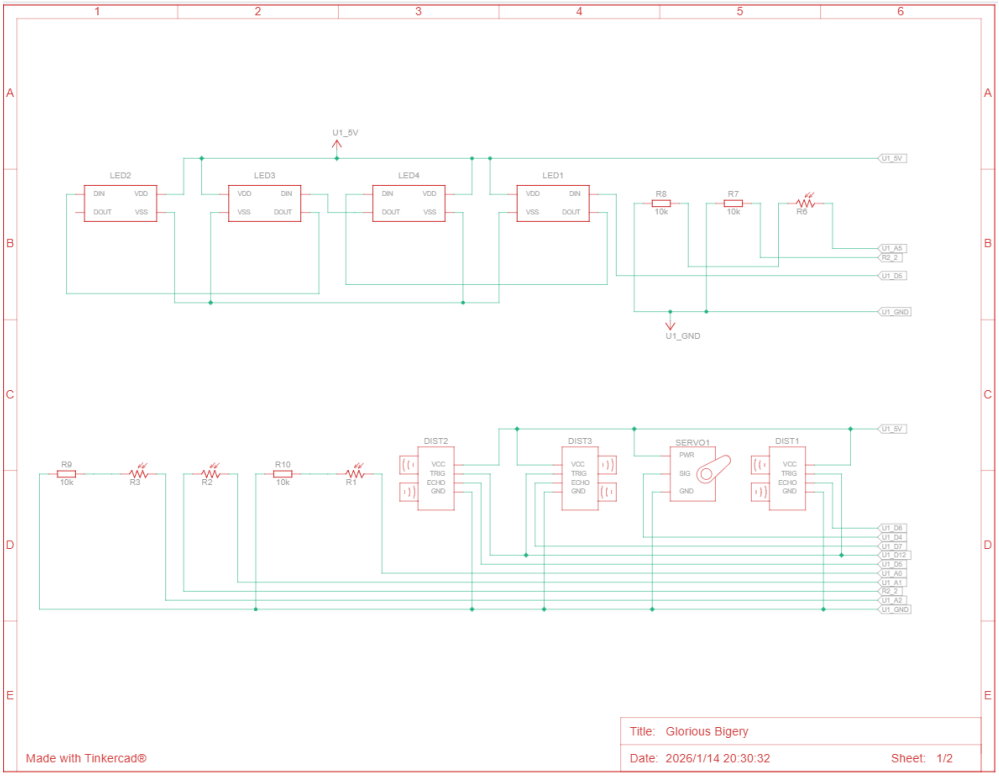


Figure 6 - BB046 | Schematic View 2

Separation Between Hardware, Software, and Behavior

The project structure reflects the architectural philosophy of the robot.

Hardware Modules (/common/robot/)

This folder contains all low-level modules responsible for controlling individual hardware components:

- Motors and pulse sensors;
- Ultrasonic sensors;
- Reflective sensor array;
- Gripper servo;
- Neopixel LEDs;
- HC-12 wireless communication.

Each module exposes simple reusable functions without defining any navigation logic.

Reusable Utilities (/common/tools/ and /common/constant/)

These folders contain:

- Pin definitions and robot constants;
- Timer utilities based on millis();
- Debugging tools;
- Macros and helper functions.

They improve code readability and avoid duplication across the project.

Behavior Programs (/maps_program/)

This folder represents the high-level logic layer of the robot.

Each subfolder implements a complete navigation strategy:

- Follow_single_line;
- Maze_line;
- Physical_maze.

These programs combine hardware modules to create complete robot behaviors

Each program contains:

- Setup function;
- A main execution function repeatedly called from loop ().

Entry Point (main.cpp)

The main.cpp file is the starting point of the robot program.

Its responsibilities are minimal by design:

- Initialize communication and LEDs;
- Select which maze program to run;
- Call the corresponding setup and loop functions.

This keeps the entry point clean while all logic is delegated to specialized modules.

Communication and Monitoring Architecture

Beyond autonomous navigation, the robot was designed to transmit real-time operational data.

The communication flow is as follows:

1. Robot sensors and status data are sent via the HC-12 wireless module.
2. Data is received by an Arduino Uno R4 Wi-Fi base station.
3. The base station forwards the data to a proxy server.
4. The server distributes data to:
    a. A backend system for database storage.
    b. A frontend dashboard via WebSocket for live monitoring.

This architecture allows real-time visualization of:

- Reflective sensor readings;
- Sonar distances;
- Neopixel states;
- Gripper status.

Architectural Benefits

This architectural approach provides several advantages:

- Code reuse across different robots;
- Easy testing of individual components;
- Clear debugging through isolated modules;
- Flexibility to add new may strategies without modifying hardware code;
- Clean separation between sensing, activation, logic, and communication.

User Stories

As a user, I want to see three different robots operate in three different maze types (line following, line maze, and physical maze), so that each robot can demonstrate a specific navigation capability.

As a user, I want each robot to remain in a parking area and wait for a start signal (e.g. HC-12 wireless signal), so that all robots begin their tasks in a controlled and coordinated manner.

As a user, I want the robots to navigate to a black target area and use their grippers to pick up an object before entering the maze, so that the required mission sequence is completed correctly.

As a user, I want the robots to turn left and enter the maze after successfully picking up the object, so that they follow the predefined course layout.

As a user, I want the Neopixel LEDs on each robot to indicate the robot's current movement direction, so that I can easily understand what the robot is doing at any time.

As a user, I want robots to detect and avoid obstacles using their sensors, particularly in the line-following and line-maze sections, so that collisions are avoided and the robots can continue safely.

As a user, I want the robots' grippers to securely hold the object throughout the entire maze, so that the payload is not dropped during operation.

As a user, I want robots to transmit sensor and status data (including sonar distances, reflective sensor values, gripper status, and LED state) to a database, so that the data can be stored for debugging and analysis.

As a user, I want to view detailed real-time sensor information on a website, including reflective sensor readings, front/left/right sonar distances, LED colors, and gripper status, so that the robot behavior can be monitored remotely, and its current condition is clearly understood.

As a user, I want robots to place the object in the final black target area and then stop moving, so that the task is completed successfully and safely.

As a user, I want robots to communicate with each other using the HC-12 wireless module, so that one robot can trigger the next robot to start its operation in sequence.

Hardware Design

The robot integrates multiple hardware components that work together to enable autonomous navigation across different maze scenarios. Each component was selected to provide specific sensory input or activation capability required for a line tracking obstacle detection, movement control, and communication.

Motors and Pulse Sensors (LM393)

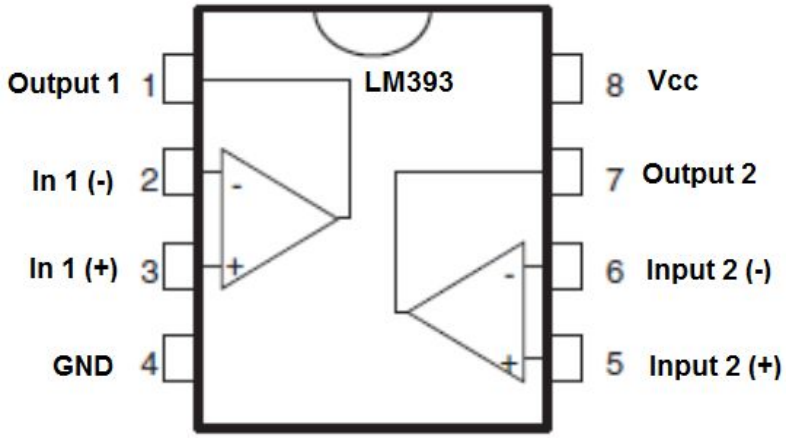


Figure 7 - Motors

The robot uses two DC motors to control movement. To ensure precise motion and enable speed correction, each motor is equipped with an LM393 optical speed sensor.

This sensor generates digital pulses whenever the motor shaft rotates. By counting these pulses using hardware interruptions, the robot can measure wheel rotation in real time. This information is essential for:

- Keeping the robot moving straight;
- Applying PID correction;
- Measuring distance travel;
- Executing precise turns.

The pool sensors are connected to interrupt capable pins, allowing accurate counting even at high speeds.

Why is this important?

Without pulse feedback, the robot cannot guarantee straight movement due to natural differences between the motors.

Parameter	Specification
Output Types	Digital (D0) and Analog (A0)
Output Indicator	On-board LED indicating output pulses on D0
Operating Voltage	3.3 V - 5 V

Table 2 - Motors | Specification

(/common/robot/motor/motor.cpp)

```cpp
#include <Arduino.h>
#include "common/constant/board.h"
#include "common/constant/motor.h"
#include "common/tools/Timer.h"
#include "common/robot/globalvariable/globalVariable.h"
Timer timestampCountPulsesRight;
Timer timestampCountPulsesLeft;
/**
 * @name countLeftPulses
 * @author Fumbre (Vladyslav)
 * @date 13-11-2025
 * @details count pulses for left motor
*/
void countLeftPulses() {
  if (timestampCountPulsesLeft.executeOnce(0)) {
    motor_left_pulses_counter++;
  }
  setupPulseCounter();
}
/**
 * @name countRightPulses
 * @author Fumbre (Vladyslav)
 * @date 13-11-2025
 * @details count pulses for right motor
*/
void countRightPulses() {
  if (timestampCountPulsesRight.executeOnce(0)) {
    motor_right_pulses_counter++;
  }
  setupPulseCounter();
}
/**
 * @name setupMotor
 * @author Fumbre (Vladyslav)
 * @date 13-11-2025
 * @details configurate pins for motor
*/
void setupMotor() {
  for (int i = 0; i < PINS_MOTOR_LENGTH; i++) {
    pinMode(PINS_MOTOR[i], OUTPUT);
    digitalWrite(PINS_MOTOR[i], LOW);
  }
  pinMode(PIN_MOTOR_LEFT_PULSE, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_MOTOR_LEFT_PULSE), countLeftPulses, RISING);
  pinMode(PIN_MOTOR_RIGHT_PULSE, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_MOTOR_RIGHT_PULSE), countRightPulses, RISING);
};
```

Ultrasonic Sensor - HC-SR04

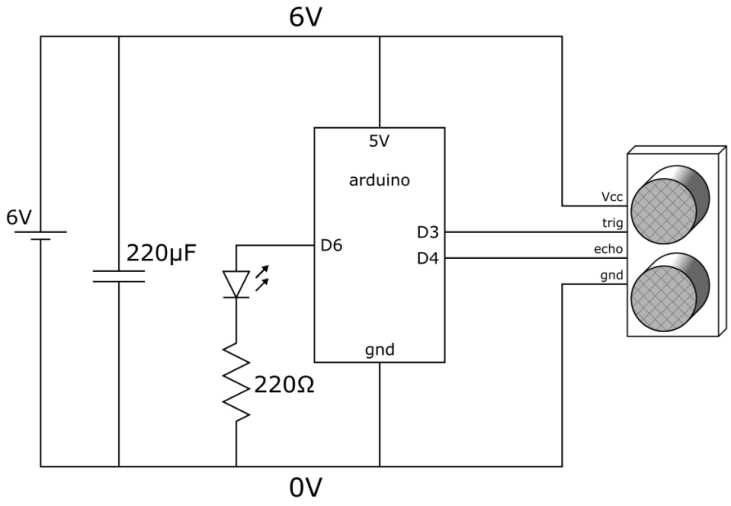


Figure 8 - Sonar

The HC-SR04 ultrasonic sensor is used for distance measurement and is essential for the physical mass scenario.

It works by emitting high frequency sound pools and measuring the time taken for the echo to return. From these, the distance to obstacles is calculated.

The sonar is used to:

- Detect walls in front of the robot.
- Check left and right paths (BB011 configuration).
- Make navigation decisions in the physical maze.

A timeout of 15 milliseconds is used to speed up reading while still covering a sufficient range for amazing navigation.

Parameter	Specification
Operating voltage	+5 V
Measuring distance	2 cm - 400 cm
Reliable measuring range	2 cm - 200 cm
Accuracy	±3 mm
Measuring angle	< 15º
Operating current	< 15 mA
Operating frequency	40 kHz

Table 3 - Sonar | Specification

```
long measureDistance() {
    // Send 10µs pulse to trigger
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    // Measure the echo time with timeout
    long duration = pulseIn(ECHO_PIN, HIGH, 15000);
    // Convert time to distance (cm)
    long distance = duration * 0.034 / 2;
    return distance;
}
```

Reflective Sensor Array (8-Channel Infrared)

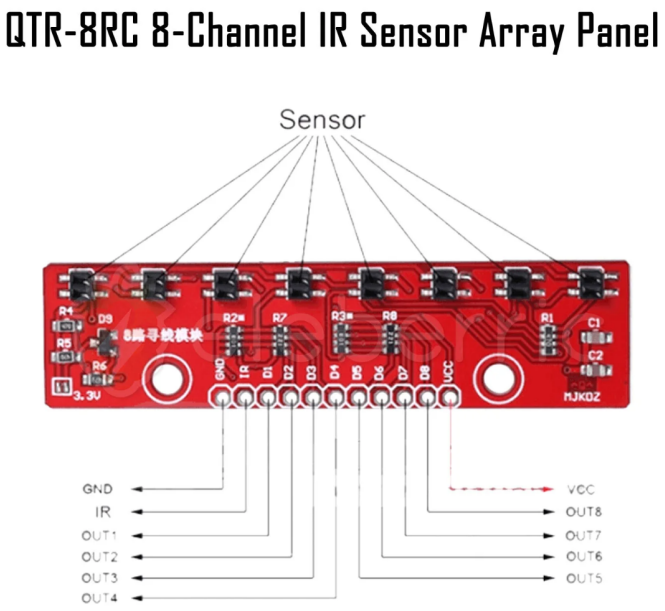


Figure 9 - Reflective Sensor

The reflective sensor array is the core component for both the line following and line maze challenges.

It consists of eight infrared sensors that detect differences between reflective (white) and non-reflective (black) surfaces.

The sensor is used to:

- Detect the position of the line under the robot.
- Identify intersections and turning points.
- Determine if the robot is centered, slightly off, or fully off the line.

A calibration system was implemented to distinguish between the initial surface and the black line using thresholds and margins.

Why is this critical?

This sensor allows the robot to interpret the ground as a binary pattern, which is later translated into navigation decisions.

Parameter	Specification
Operating Voltage	3.3 V - 5 V
Output Type	Digital
Optimal Sensing Distance	3 mm
Maximum Recommended Sensing Distance	9.5 mm
Number of Channels	8 Infrared Sensors
Pinout	GND: Ground VCC: Input voltage IR: Infrared enable D1–D8: Sensor outputs

Table 4 - Reflective Sensor | Specification

```
/**
 * @name ReflectiveSensor
 * @authors Fumbre (Vladyslav) & Aria & Sunny & Uraib
 * @date 08-12-2025
 */
#include <Arduino.h>
#include "common/tools/ARRAY_SIZE.h"
#include "common/constant/reflective_sensor.h"
/**
 * Bitmask tables used to classify line position
 */
// Centered on the line
const uint8_t centerPatterns[] = {
    0b00011000,
    0b00111100
};
// Slight deviation to the left
const uint8_t slightLeftPatterns[] = {
    0b01100000,
    0b00110000,
    0b01110000
};
// Slight deviation to the right
const uint8_t slightRightPatterns[] = {
    0b00000110,
    0b00001100,
    0b00001110
};
/**
 * @enum LineState
 * @details Represents the robot position relative to the line
 */
```

```
enum LineState {
    CENTER,
    SLIGHT_LEFT,
    SLIGHT_RIGHT,
    HARD_LEFT,
    HARD_RIGHT,
    ALL_WHITE,
    ALL_BLACK,
    OTHER
};
/**
 * @name detectLineState
 * @authors Uraib
 * @date 04-12-2025
 * @param sensorPattern Binary pattern read from the reflective sensor array
 * @return LineState Current detected line state
 * @details Matches the sensor pattern against predefined bitmasks
 *          to determine the robot position relative to the line.
 */
LineState detectLineState(uint8_t sensorPattern) {
    for (uint8_t i = 0; i < ARRAY_SIZE(centerPatterns); i++) {
        if (sensorPattern == centerPatterns[i]) {
            return CENTER;
        }
    }
    for (uint8_t i = 0; i < ARRAY_SIZE(slightLeftPatterns); i++) {
        if (sensorPattern == slightLeftPatterns[i]) {
            return SLIGHT_LEFT;
        }
    }
    for (uint8_t i = 0; i < ARRAY_SIZE(slightRightPatterns); i++) {
        if (sensorPattern == slightRightPatterns[i]) {
            return SLIGHT_RIGHT;
        }
    }
    if (sensorPattern == 0b00000000) {
        return ALL_WHITE;
    }
    if (sensorPattern == 0b11111111) {
        return ALL_BLACK;
    }
    return OTHER;
}
```

Gripper Servo - SG90

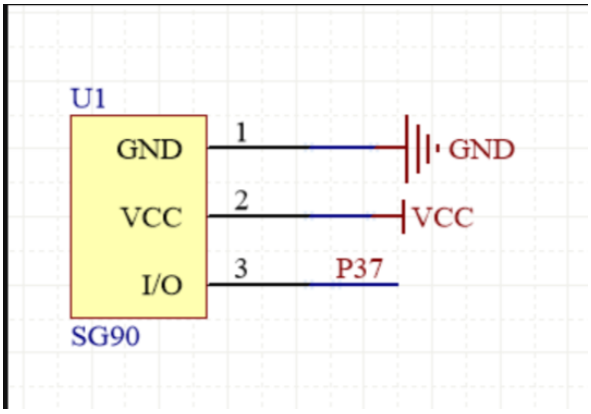


Figure 10 - Gripper

The SG90 Micro server motor controls the robotic repair mounted on the robot.

The servo is controlled using software generated PWM signals. By varying the pulse width between 1000 microseconds and 2000 microseconds, the gripper cannot be fully closed or fully opened.

The server logic is designed to be non-blocking, allowing the robot to continue navigation while maintaining the server position.

Parameter	Specification
Control Signal	PWM (Pulse Width Modulation)
Neutral Position	0° (±1.5 ms pulses)
Left Position	+90° (±2 ms pulses)
Right Position	-90° (±1 ms pulses)
Modulation	Analog
Rotational Range	180°
Operating Voltage	4.8 V - 6.0 V
Operating Speed	0.12 s / 60° (4.8 V); 0.11 s / 60° (6.0 V)

Table 5 - Gripper | Specification

```
/**
 * @name setupGripper
 * @authors Francisco
 * @date 3-12-2025
 * @details Configures the gripper control pin and ensures
 *          the servo signal starts in a safe LOW state.
 */
void setupGripper() {
    pinMode(GRIPPER, OUTPUT);
    digitalWrite(GRIPPER, LOW);
}
/**
 * @name gripper
 * @authors Francisco
 * @date 3-12-2025
 * @details Generates non-blocking servo control pulses in software
 *          to control a gripper.
 * @details Maintains a 20 ms interval between pulses, ensuring
 *          correct servo operation.
 * @param pulseMicroseconds Pulse width in microseconds that sets
 *        the gripper position.
 *        - Typical range for standard servos:
 *          1000 µs (fully closed) to 2000 µs (fully open).
 *        - If pulseMicroseconds <= 0, the last set pulse is maintained.
 * @note This function should be called repeatedly in the main loop
 *       to continuously control the servo.
 */
void gripper(int pulseMicroseconds) {
    static unsigned long nextPulseTime = 0;
    static int lastPulse = 2000;
    if (pulseMicroseconds > 0) {
        lastPulse = pulseMicroseconds;
    }
    unsigned long now = millis();
    if (now >= nextPulseTime) {
        nextPulseTime = now + 20;
        digitalWrite(GRIPPER, HIGH);
        delayMicroseconds(lastPulse);
        digitalWrite(GRIPPER, LOW);
    }
}
/**
 * @name gripperCatch
 * @authors Francisco
 * @date 3-12-2025
 * @details Closes the gripper.
 */
void gripperCatch() {
    gripper(1000);
    gripperStatus = true;
}
/**
 * @name gripperUnCatch
 * @authors Francisco
 * @date 3-12-2025
 * @details Opens the gripper.
 */
void gripperUnCatch() {
    gripper(2000);
    gripperStatus = false;}
```

Neopixel LEDs

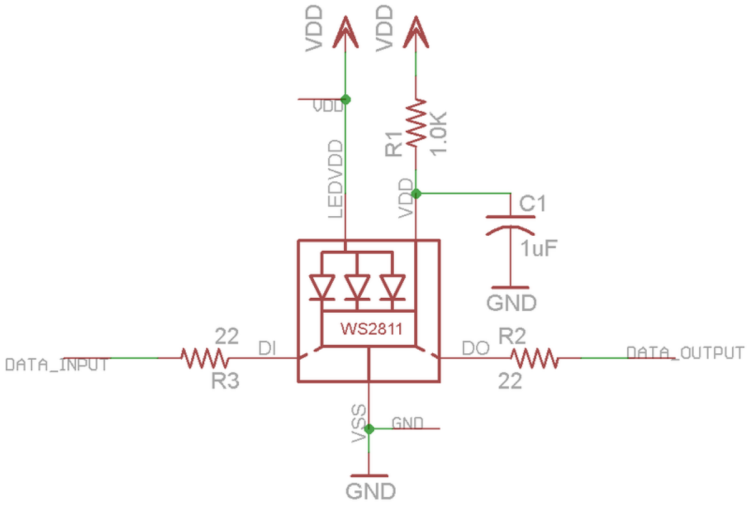


Figure 11 - Neopixel

The robot includes 4 Neopixel RGB LEDs used for visual feedback.

These lads are individually addressable and allow the robot to display status information such as:

- Direction indication.
- Operational state.
- Debugging signals during development.

The leads are controlled through a single digital pin using the Adafruit NeoPixel library.

Why these matters?

They provide immediate visual insight into robot decisions without needing serial monitoring.

Parameter	Specification
Supply voltage (VCC)	5 V (or external 5 V power supply)
Ground (GND)	Common ground (must be shared with the controller)
Data Input (DIN)	Digital input pin (e.g., D6)
Communication Type	Single-wire digital data

Table 6 - Neopixel | Specification

```
/**
 * @name interface of Neopixel
 * @author Sunny
 * @date 24-11-2025
*/
#pragma once
#include "common/constant/neopixel.h"
// #include "common/tools/Neopixel.h"
#include "common/robot/globalvariable/globalVariable.h"
#include <Adafruit_NeoPixel.h>
void initNeopixelPins();
void turnOnAllLeds(int R, int G, int B);
void turnOffAllLeds();
void turnOnSomeLeds(int index[], int length, int R, int G, int B);
void turnOffSomeLeds(int index[], int length);
void turnOnOneLed(int index, int R, int G, int B);
```

```
/**
 * @name functions of neopixels
 * @author Sunny
 * @date 24-11-2025
 */
#include "neopixel.h"
// define Neopixel class
// Neopixel pixel(PIN_NI, NUMBER_OF_NEOPIXEL);
Adafruit_NeoPixel strip(NUMBER_OF_NEOPIXEL, PIN_NI, NEO_RGB + NEO_KHZ800);
/**
 * @name initNeopixelPins
 * @author Sunny
 * @date 24-11-2025
 */
void initNeopixelPins()
{
  strip.begin();
  strip.clear();
}
/**
 * @name initNeopixelPins
 * @author Sunny
 * @date 26-11-2025
 * @param R red color value (0-255)
 * @param G green color value (0-255)
 * @param B blue color value (0-255)
 */
void turnOnAllLeds(int R, int G, int B)
{
  for (int i = 0; i < 4; i++)
  {
    strip.setPixelColor(i, strip.Color(R, G, B));
    colorArray[i][0] = R;
    colorArray[i][1] = G;
    colorArray[i][2] = B;
  }
  strip.show();
}
/**
 * @name turnOffAllLeds
 * @author Sunny
```

```
 * @date 26-11-2025
 */
void turnOffAllLeds()
{
  for (int i = 0; i < 4; i++)
  {
    strip.setPixelColor(i, strip.Color(0, 0, 0));
    colorArray[i][0] = 0;
    colorArray[i][1] = 0;
    colorArray[i][2] = 0;
  }
  strip.show();
}
/**
 * @name turnOnSomeLeds
 * @author Sunny
 * @date 26-11-2025
 * @param index array of led index
 * @param length length of led index array
 * @param R red color value (0-255)
 * @param G green color value (0-255)
 * @param B blue color value (0-255)
 */
void turnOnSomeLeds(int index[], int length, int R, int G, int B)
{
  for (int i = 0; i < length; i++)
  {
    strip.setPixelColor(index[i], strip.Color(R, G, B));
    colorArray[index[i]][0] = R;
    colorArray[index[i]][1] = G;
    colorArray[index[i]][2] = B;
  }
  strip.show();
}
/**
 * @name turnOffSomeLeds
 * @author Sunny
 * @date 26-11-2025
 * @param index array of led index
 * @param length length of led index array
 */
void turnOffSomeLeds(int index[], int length)
{
  for (int i = 0; i < length; i++)
  {
    colorArray[index[i]][0] = 0;
    colorArray[index[i]][1] = 0;
    colorArray[index[i]][2] = 0;
    strip.setPixelColor(index[i], strip.Color(0, 0, 0));
  }
  strip.show();
}
void turnOnOneLed(int index, int R, int G, int B)
{
  strip.setPixelColor(index, strip.Color(R, G, B));
  strip.show();
}
```

HC-12 Wireless Communication Module

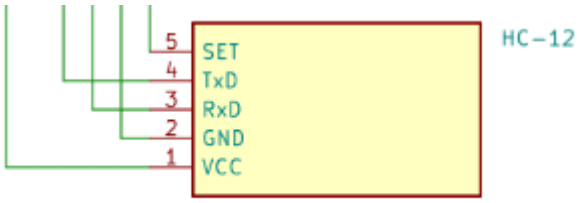


Figure 12 - HC12

The HC-12 Module provides long-range wireless serial communication between the robot and the base station.

It operates in the 433 MHz band and allows the robot to transmit real time data such as:

- Sensor readings.
- LEDs states.
- Gripper status.
- Sonar distances.

This data is later forwarded to a dashboard and stored in a database.

Why this is important?

This transforms the robot from an isolated system into a monitored robotic platform.

Parameter	Specification
Operating Voltage	3.2 V - 5 V
Data Type	Digital Seria Data
Communication Range	< 100 m (typical); up to 1,000 m in open space
Operating Frequency	433 MHz
Pinout	VCC – Power input GND – Common ground RXD – Serial data input (weak pull-up) TXD – Serial data output SET – Configuration input ANT – 433 MHz antenna pin GND – Common ground GND – Common ground NC – No connection ANT1 – IPEX20279-001E-03 antenna socket ANT2 – 433 MHz spring antenna solder pad

Table 7 - HC12 | Specification

```
/**
 * @name interface of HC-12
 * @author Sunny
 * @date 15-12-2025
 */
#pragma once
#include "common/constant/hc12.h"
#include <Arduino.h>
#include <SoftwareSerial.h>
extern SoftwareSerial hc12;
void buildHC12Connection();
void sendDataFromHC12(String data);
String receiveDataFromHC12();
```

```
#include "hc12.h"
/**
 * Create a SoftwareSerial object to communicate with the HC-12 module
 */
SoftwareSerial hc12(RX, TX);
/**
 * @name buildHC12Connection
 * @author Sunny
 * @date 15-12-2025
 * @details Builds the serial connection between the HC-12
 *          wireless module and the Arduino.
 */
void buildHC12Connection() {
    hc12.begin(9600);
```

```
}
/**
 * @name sendDataFromHC12
 * @author Sunny
 * @date 15-12-2025
 * @param data String message to be transmitted
 * @details Sends formatted data through the HC-12 module.
 */
void sendDataFromHC12(String data) {
    hc12.println(data);
}
/**
 * @name receiveDataFromHC12
 * @author Sunny
 * @date 15-12-2025
 * @return String Data received from HC-12
 * @details Reads incoming data from the HC-12 module if available.
 */
String receiveDataFromHC12() {
    String receivedData = "";
    if (hc12.available()) {
        receivedData = hc12.readStringUntil('\n');
    }
    return receivedData;
}
```

Summary of Hardware Contribution

Each hardware component plays a distinct role:

Component	Role in the Robot
Motors + LM393	Precise movement and PID correction
HC-SR04	Obstacle detection and wall navigation
Reflective sensor	Line detection and intersection logic
SG90 Gripper	Controlled actuation via PWM
Neopixels	Visual feedback and debugging
HC-12	Wireless data transmission

Table 8 - Summary of Hardware Contribution

Together, these components provide the sensory input and actuation capabilities required for the robot to operate autonomously across all maze scenarios.

Software Architecture

The software was designed with a strong emphasis on modularity, code reuse, and clear separation of responsibilities.

Rather than implementing all logic inside a single file, the project follows a layered structure where each model is responsible for a specific aspect of the robot's operation.

This architectural approach improves readability, maintainability, and scalability, allowing new behaviors or hardware configurations to be added without affecting the core system.

Development Environment

The project was developed using:

- Visual Studio Code as the main development environment.
- PlatformIO for building management, dependency handling, and firmware upload.
- Arduino framework for low-level hardware interaction.

This setup provides a professional workflow with version control support, code navigation, and cross-platform compatibility.

Project Folder Structure

The software is organized into a clear and consistent directory structure, separating reusable models from behavior-specific logic.

/common/

The /common/ directory contains all reusable components shared across different robot configurations and missing programs.

/common/constant/

This folder defines all constant values used throughout the project, including:

- Pin mappings for different robot boards.
- Motor configuration constants.
- Sensory related constants.
- Communication parameters.

By centralizing constants, hardware changes can be applied without modifying the main logic.

/common/robot/

This folder contains low level hardware abstraction modules, each responsible for a specific component:

- motor/ - motor control and pulse counting.
- movement/ - higher level movement functions based on pulse feedback.
- sonar/ - ultrasonic distance measurement.
- reflective_sensor/ - line detection and pattern classification.
- gripper/ - servo control logic.
- neopixel/ - let's state management.
- hc12/ - wireless communication.
- datasend/ - structure data transmission.

Each module exposes a clean interface and doesn't contain navigation logic.

/common/tools/

The tools/ folder provides utility functionality used across the project, such as:

- Non-blocking timers based on millis();
- Helper macros and rationalities.
- Debugging and execution control tools.

These utilities prevent code duplication and support a non-blocking execution.

/common/robot/globalvariable/

This folder contains global variables that use for sending to the Arduino Uno R4 or shared in the project:

- Motor pulse counters.
- Sensor readings.
- Neopixel colour states.
- Gripper status.

All shared states are clearly defined and centralized, improving traceability and debugging.

Behavior Programs (/maps_program/)

The /maps_program/ directory represents the behavioral layer of the robot.

Each subfolder implements a complete navigation strategy:

- Line following;
- Line maze;
- Physical maze.

Each behavior program includes:

- A setup function for initialization.
- A loop function that is repeatedly executed.

These programs combine reusable hardware modules to create complete autonomous behaviors.

Entry Point - /main.cpp

The main.cpp file act has the single-entry point of the application.

Its responsibilities are intentionally minimal:

- Initialize hardware modules;
- Establish communication;
- Select which behavior program to execute;
- Call the corresponding setup and loop functions.

This design keeps the entry point clean and avoids embedding logic that belongs in specialized modules.

Board Configuration Using Preprocessor Directives

To support multiple robot boards (BB011, BB016, BB046), the software uses preprocessor directives.

By defining the target board at compile time:

- Pin assignments are selected automatically;
- Available sensors are enabled or disabled;
- Board-specific behavior is handled transparently.

This approach allows the same code base to run on different hardware configurations without modification.

Navigation Logic

The navigation logic of the robot defines how sensor data is transformed into movement decisions.

Rather than relying on a single navigation strategy, the robot dynamically applies different control logics depending on the maze scenario.

All navigation algorithms are implemented using non-blocking logic, allowing the robot to continuously read sensors, update movement, entries meet data without halting execution.

Line Following

Line following is based on continuous feedback from the 8-channel reflective sensor array.

Sensor Patter Interpretation

Each sensor reading is converted into a binary pattern representing the presence or absence of a line under each sensor.

These patterns are matched against predefined masks to determine the robot's relative position to the line.

- Centered on the line.
- Slightly left or right.
- hard deviation.
- Fully off the line.

This abstraction allows the robot to make movement decisions without relying on raw sensor values.

Motor Correction Strategy

Based on the detected line state, the robot adjusts the motor speeds:

- Centered - Both motors run at equal speed.
- Slight deviation - one motor slows down to correct direction.
- Hard deviation - aggressive correction to recenter the robot.

Motor pulse feedback is used to stabilize movement and avoid drift.

Line Maze

The line maize extends the line following behavior by introducing intersections and decision points.

Intersection Detection

Intersections are detected when multiple reflective sensors simultaneously detect a black surface.

Specific bitmask patterns are used to identify:

- Left turn opportunities.
- Right turn opportunities.
- Cross intersections.

This allows the robot to distinguish between normal line following and maze decision points.

Timed Turning Logic

Turns are executed using time-based movement controlled by non-blocking timers:

- Motors are set to opposite speeds to rotate the robot;
- The rotation duration is controlled using millis();
- After the turn, normal line following logic resumes.

This approach avoids blocking delays and ensures continuous sensor updates during navigation.

Physical Maze

In the physical maze scenario, no line guidance is available. Navigation relies entirely on ultrasonic sensors and decision-making logic.

Obstacle Detection

The robot continuously measures the distance to surrounding obstacles:

- Front sonar is used to detect walls directly ahead.
- Side sonars (when available) detect lateral openings.

Distances below a defined threshold are considered obstacles.

Decision-Making Strategy

The physical maze logic follows A deterministic rule set:

1. Move forward while no obstacle is detected in front.
2. When the front obstacle is detected:
    a. Turn left if the left side is free.
    b. Else, turn right if the right side is free.
    c. Else, perform 180° turn.

This strategy ensures the robot can navigate unknown environments without prior mapping.

Movement Execution

Turns and movements are executed using:

- Motor speed control.
- Pools counting for movement consistency.
- Non-blocking timers for precise rotation durations.

After completing a turn, the robot resumes forward motion and repeats the decision cycle.

Non-Blocking Execution and Stability

All navigation behaviors are implemented using non-blocking logic. (/common/tools/Timer.h)

- No delay() calls inside navigation loops.
- Timers based on millis() control movement duration.
- Sensors are continuously updated during motion.

This ensures smooth navigation comma responsive obstacle detection comma and reliable data transmission.

Communication System

The communication system was designed to enable real-time data transmission, monitoring, and post-run analysis.

Instead of operating as a standalone robot, the system integrates wireless communication, a base station, a backend server, and a visual dashboard.

This architecture allows sensor data and robot states to be observed externally while the robot is operating autonomously.

Network Diagram

The overall communication flow is summarized in the network diagram below:

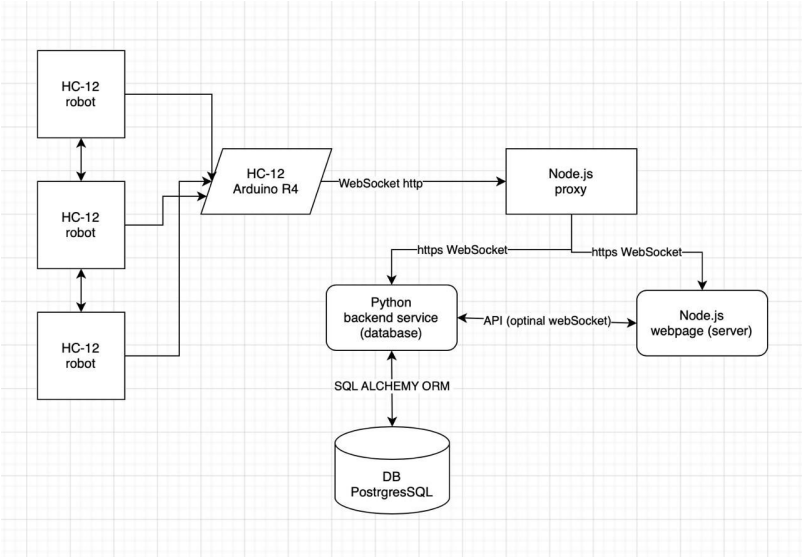


Figure 13 - Network Diagram

This diagram illustrates the full path from the robot to the dashboard.

Communication Overview

The communication system follows a multistage see through architecture:

1. The robot collects sensors and status data.
2. Data is transmitted wirelessly using the HC-12 module to Arduino R4.
3. A base station receives and forwards the data to the proxy.
4. The proxy sends data to frontend and backend parts
5. A backend server processes and stores the data in database.
6. A frontend dashboard displays live information via web socket.

Robot-Side Communication (HC-12)

On the robot side, the HC-12 wireless module is used to transmit structured serial messages.

Data Types Transmitted

The robot periodically sends:

- Reflective sensor readings.
- Ultrasonic sensor distances.
- Neopixel LED states.
- Gripper status.

All data is formatted as structured strings to simplify parsing downstream.

Transmission Control

Data transmission is controlled using nonblocking timers, ensuring that:

- Navigation logic is never interrupted.
- Sensor reading remains continuous.
- Communication frequency is regulated.

This allows reliable real time monitoring without affecting robot behavior.

WEB

Base Station Architecture

The base station is implemented using an Arduino Uno R4 Wi-Fi. The Arduino parses and sends data to the local server (proxy). Meanwhile proxy sends the data further to backend (python for database) and frontend (node.js for hosting frontend) servers

Its responsibilities include:

- Receiving HC 12 wireless data.
- Parsing incoming messages.
- Forwarding the data via web socket to proxy.

The base station act has a bridge between the robot and the networked systems.

Python Backend System

The backend server processes and stores the robot data in Postgres database.

Responsibilities

The backend is responsible for:

- Receiving data from the proxy.
- Validating and parsing messages.
- Storing data in a structured database.
- Providing real time data streams.

Frontend Dashboard

The frontend provides a visual interface for monitoring the robot in real time using web socket. Node.js part is a client and server in the same time. It can get data using python API to get access to database (useful for first loading page) or get the message from Proxy or Python and send it to all own clients (ex.: client side JavaScript part), while JavaScript will update the page in real life.

Live Monitoring

The dashboard displays:

- Reflective sensor states.
- Sonar distance values.
- Neopixel colour information.
- Gripper open/close state.

Data is updated in real time using WebSocket communication



Figure 14 - Webpage

User Feedback and Debugging

The dashboard serves not only has a visualization tool, but also has a debugging aid:

- immediate detection of sensor faults.
- verification of navigation decisions.
- insight into robot behavior during execution.

Database and Data Model

All received data is stored in a database for later analysis.

Data Storage

The database stores:

- Timestamped sensor readings;
- Robot identification data;
- Execution states.

Entity-Relationship Diagram (ERD)

An entity relationship diagram was designed to model the relationships between:

- Robots;
- Sensor data entries;

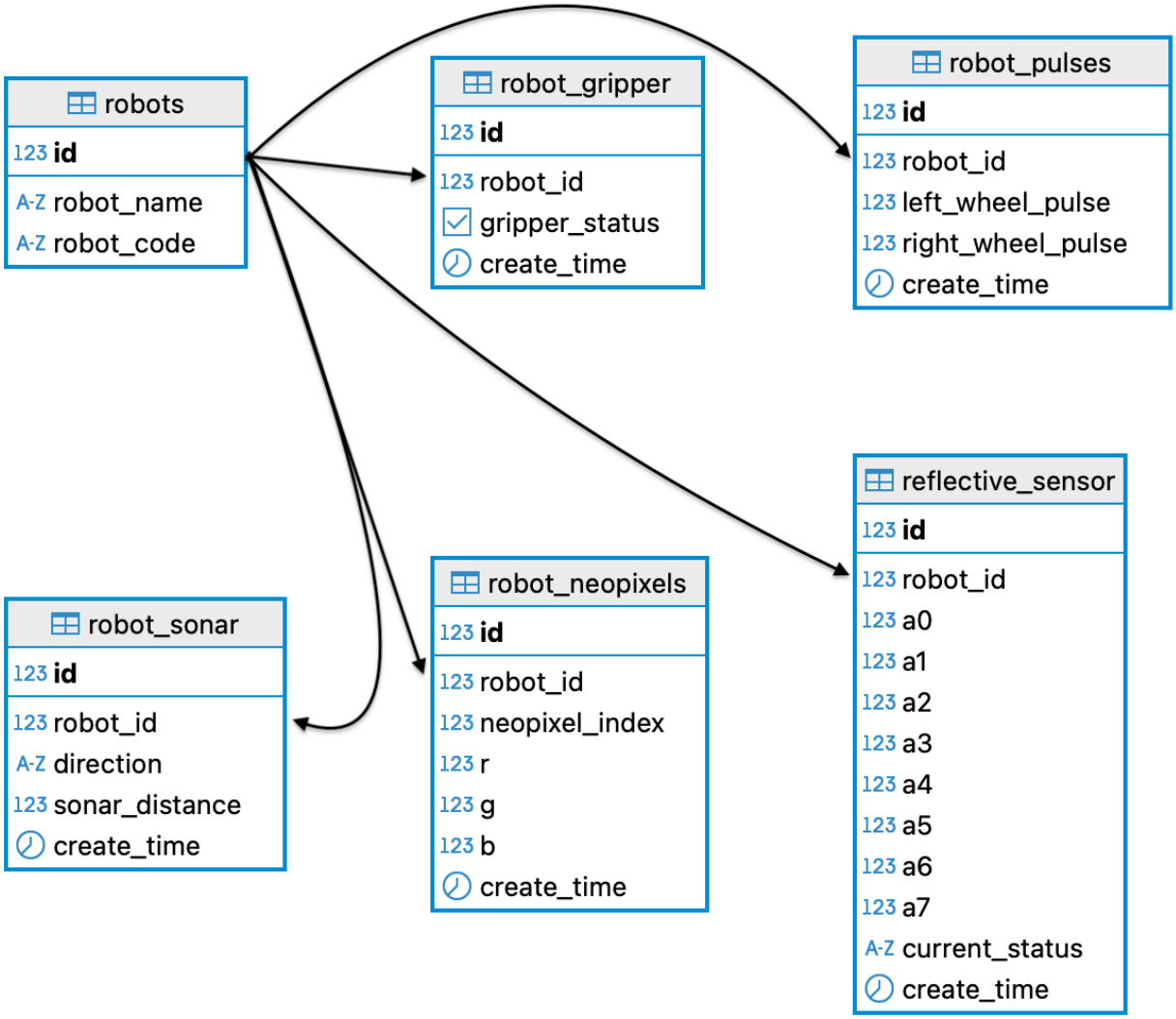


Figure 15 - Entity-Relationship Diagram (ERD)

Testing and Measurements

Testing and validation were essential throughout the development of the project to ensure reliability, stability, and correct behavior across all may scenarios.

Rather than testing only the final system, individual components and subsystems were evaluated incrementally.

Testing Strategy

The testing strategy followed a bottom up approach:

1. Individual hardware components were tested in isolation.
2. Low level software modules were validated independently.
3. Integrated behaviors were tested under realistic conditions.

Advisory Report

The advisory report documents feedback received during the development process and the actions taken to improve the system.

Identified Issues

During advisory evaluations, several aspects were identified for improvement, including:

- Motor stability at higher speeds;
- Sonar reliability in narrow spaces;
- Timing accuracy during turns;
- Code structure and modularity.

Implemented Improvements

Based on the advisory feedback, the following improvements were implemented:

- Motor stabilization using pulse feedback;
- Optimization of sonar timeout values;
- Replacement of blocking delays with non-blocking timers;
- Further modularization of software components

Measurement Report

The measurement report focuses on quantitative evaluation of the robots performance.

Performance Metrics

The following metrics were measured:

- Sonar distance accuracy;
- Response time to obstacles;
- Consistency of motor movements;
- Stability during line following.

Results and Analysis

The collected data demonstrated that:

- Sonar measurements remained within acceptable error margins;
- Motor pulse feedback improved straight line motion;
- Non-blocking control reduced response delays;
- Navigation logic remains stable across repeated runs

Integrated System Testing

After individual validation, the full system was tested in all my scenarios:

- Line following;
- Line maze;
- Physical mice.

Each scenario was completed multiple times to verify consistency and repeatability.

a0	a1	a2	a3	a4	a5	a6	a7	current_status	create_time
522	490	481	357	289	387	969	985	00000011	2026-01-14 10:23:44.193943
734	989	911	414	344	402	395	499	01100000	2026-01-14 10:23:42.271087
990	986	576	433	388	446	439	551	11000000	2026-01-14 10:23:39.853975
537	515	513	410	752	984	524	515	00001100	2026-01-14 10:23:37.908129
566	525	511	383	308	538	972	976	00000011	2026-01-14 10:23:35.762941
554	528	547	967	953	439	399	494	00011000	2026-01-14 10:23:33.302519
537	519	929	981	392	394	368	471	00110000	2026-01-14 10:23:31.950127
539	515	513	416	904	987	814	509	00001110	2026-01-14 10:23:28.999525
590	984	963	429	357	406	388	485	01100000	2026-01-14 10:23:27.583861
431	420	444	335	283	372	372	487	00000000	2026-01-14 10:23:24.597656

Table 9 - Reflective Sensor | BB016

PS: 0 => front, 2 => left, 1 => right

Direction	Sonar Distance	Create Time
2	0	2026-01-14 10:23:44.477924
0	34.59	2026-01-14 10:23:44.477924
1	0	2026-01-14 10:23:44.477924
2	0	2026-01-14 10:23:42.285536
0	64.36	2026-01-14 10:23:42.285536
1	0	2026-01-14 10:23:42.285536
0	400	2026-01-14 10:23:40.231345
1	0	2026-01-14 10:23:40.231345
2	0	2026-01-14 10:23:40.231345
1	0	2026-01-14 10:23:37.922684
0	400	2026-01-14 10:23:37.922684
2	0	2026-01-14 10:23:37.922684
0	97.17	2026-01-14 10:23:35.777162
2	0	2026-01-14 10:23:35.777162
1	0	2026-01-14 10:23:35.777162
2	0	2026-01-14 10:23:33.610746
1	0	2026-01-14 10:23:33.610746
2	0	2026-01-14 10:23:44.477924
0	34.59	2026-01-14 10:23:44.477924

Table 10 - Sonar - BB016

PS: (0,3) => LEFT, (1,2) => RIGHT

neopixel_index	R	g	b	create_time
0	0	0	0	2026-01-14 10:23:42.995009
2	255	255	0	2026-01-14 10:23:42.995009
3	0	0	0	2026-01-14 10:23:42.995009
1	255	255	0	2026-01-14 10:23:42.995009
1	0	0	0	2026-01-14 10:23:40.675914
3	255	255	0	2026-01-14 10:23:40.675914
2	0	0	0	2026-01-14 10:23:40.675914
0	255	255	0	2026-01-14 10:23:40.675914
1	0	0	0	2026-01-14 10:23:38.522044
3	255	255	0	2026-01-14 10:23:38.522044
2	0	0	0	2026-01-14 10:23:38.522044
0	255	255	0	2026-01-14 10:23:38.522044
1	255	255	0	2026-01-14 10:23:36.582369
3	0	0	0	2026-01-14 10:23:36.582369
2	255	255	0	2026-01-14 10:23:36.582369
0	0	0	0	2026-01-14 10:23:36.582369

Table 11 - Neopixel | BB016

Version History

Version control was used throughout the development of the project to track progress, manage features, and document major milestones.

The table below summarizes the main development stages of the project, highlighting completed functionalities and contributors.

Version	Date	Status	Name	Description
0.1	05-11-2025	finished	Sunny & Vladyslav	Basic movement
0.2	14-11-2025	finished	Francisco	Sonar functions
0.3	19-11-2025	finished	Francisco	Object avoidance
0.4	25-11-2025	finished	Sunny	Basic movement with PID
0.5	28-11-2025	finished	Francisco	Gripper
0.6	05-12-2025	finished	Sunny	Neopixels
0.7	08-12-2025	finished	Aria	Reflective sensor
0.8	10-12-2025	finished	Francisco	Physical maze
0.9	11-12-2025	finished	Aria	Following line
1.0	14-12-2025	finished	Vladyslav & Aria & Uraib & Sunny	2nd version of reflective_sensor
1.1	15-12-2025	finished	Vladyslav & Aria	Final version of following line
1.2	16-12-2025	finished	Vladyslav	Sequence

1.3	17-12-2025	finished	Aria	Maze line
1.4	18-12-2025	finished	Sunny & Vladyslav & Aria	Maze line 2.0
1.5	19-12-2025	finished	Sunny	Database creation and website back-end
1.6	20-12-2025	finished	Sunny	HC-12 part
1.7	21-12-2025	finished	Vladyslav	Full-stack part for robot website
1.8	22-12-2025	finished	Sunny	Base station of Arduino.

Table 12 - Version History

Weekly Reflections

Throughout the project, weekly reflections were conducted using the What-How-What methodology.

This approach encouraged continuous evaluation of both technical progress and team collaboration.

Reflection Method

Each weekly reflection follows three guiding questions:

- What?

  What was accomplished during the week?

- How?

  How were tasks executed and what strategies were applied?

- What next?

  What improvements or next steps were identified?

Weekly Reflection Summary

The weekly reflections documented:

- Progressive development of hardware and software modules;
- Identification and resolution of technical challenges;

- Improvements in team communication and task distribution;
- Adaptation of strategies based on testing and feedback.

Detailed weekly reflections are presented below.

Weekly Reflections (What-How-What)

Week 1

What (Task)	How (Action)	What (Result/Reflection)
Basic Movements	We worked on starting the project and brainstormed about all the ideas.	We finished the Basic Movements task, showed it and signed it.

Table 13 - Week 1 Reflection

Week 2

What (Task)	How (Action)	What (Result/Reflection)
Object Avoidance	We started working with the sensor and started updating the code.	We finished the sensors task, showed it and signed it.

Table 14 - Week 2 Reflection

Week 3

What (Task)	How (Action)	What (Result/Reflection)
Stay on Track	We researched what reflective sensors are and how they detect the differences	We successfully made the robot recognize the line and correct its movement; we signed the task.

What (Task)	How (Action)	What (Result/Reflection)
	between surfaces.	

Table 15 - Week 3 Reflection

Week 4

What (Task)	How (Action)	What (Result/Reflection)
Midterm Race	We improved the code to meet the requirements to finish the race.	The Midterm Race was finished, and we got the task signed.

Table 16 - Week 4 Reflection

Week 5

What (Task)	How (Action)	What (Result/Reflection)
Connectivity (HC-12)	We learned how serial communication works and why proper wiring and matching baud rates are important.	We finished the task of the HC-12 connectivity showing it and signing it.

Table 17 - Week 5 Reflection

Week 6

What (Task)	How (Action)	What (Result/Reflection)
Data Storage (SQL)	Build base station with Arduino uno R4 WIFI for receiving data from HC-12.	In this assignment, we finished the front-end and backend part of robot website. For front-end part, we use some technologies, like Node.js, websocket, EJS, Echarts, Sass and webpack. For the back-end part, we use Python, FastAPI,

What (Task)	How (Action)	What (Result/Reflection)
	Build back-end and front-end part of robot website.	websocket, SQLALchemy, and PostgreSQL. We finished the task of the Data Storage (SQL) showing it and signing it.

Table 18 - Week 6 Reflection

Week 7

What (Task)	How (Action)	What (Result/Reflection)
Test Day and Dashboard	Test the code of line following, line maze and physical maze, start sequence, end sequence.	The line following, line maze, start sequence and end sequence are finished. But there is some logical problem in physical maze. So, we need to improve the code of physical maze.

Table 19 - Week 7 Reflection

Week 8

What (Task)	How (Action)	What (Result/Reflection)
Race Day + Project Dossier	Finish the race.	Finished the race finally. But there are some points that we can improve. In following line part, sometimes robot is out of the track. In line maze, if robot meet the junction that should turn right and crossing, it should turn right, but sometimes just go forward.

Table 20 - Week 8 Reflection

Code

Project Layout

The following directory structure represents the complete and final organization of the project code has implemented in Visual Studio Code.

```
/src
│
├── /common
│   │
│   ├── /constant
│   │   ├── board.h
│   │   ├── gripper.h
│   │   ├── hc12.h
│   │   ├── motor.h
│   │   ├── neopixel.h
│   │   ├── reflective_sensor.h
│   │   ├── robot.h
│   │   └── sonar.h
│   │
│   ├── /robot
│   │   ├── /actions
│   │   ├── /datasend
│   │   ├── /globalvariable
│   │   ├── /gripper
│   │   ├── /hc12
│   │   ├── /motor
│   │   ├── /movement
│   │   ├── /neopixel
│   │   ├── /reflective_sensor
│   │   └── /sonar
│   │
│   └── /tools
│       ├── /tests
│       ├── ARRAY_SIZE.h
│       ├── bluetooth.h
│       ├── Neopixel.h
│       ├── PID.h
│       └── Timer.h
│
├── /maps_program
│   ├── /follow_single_line
│   ├── /maze_line
│   └── /physical_maze
│
└── main.cpp
```

Main Application File

main.cpp

```
#include "common/constant/robot.h"
#include "common/tools/bluetooth.h"
```

```ino
#include <Arduino.h>
#include "common/robot/neopixel/neopixel.h"
#include "maps_pogram/maze_line/maze_line.h"
#include "maps_pogram/follow_single_line/follow_single_line.h"
#include "maps_pogram/physical_maze/physical_maze.h"

int mapInit = 1;

void setup() {
  Serial.begin(9600);
  blueTooth.begin(9600);
  initNeopixelPins();
  switch (mapInit) {
  case 1:
    followLineSetup();
    break;
  case 2:
    mazeLineSetup();
    break;
  case 3:
    physicalMazeSetup();
    break;
  }
}

void loop() {
  testSonars();
  switch (mapInit) {
  case 1:
    followLine();
    break;
  case 2:
    mazeLine();
    break;
  case 3:
    physicalMaze();
    break;
  }
}
```

Common Modules

/common/constant/

board.h

(/common/constant/board.h)

```
/**
 * @name constant of robot
 * @author Fumbre (Vladyslav)
 * @date 11-11-2025
*/
#pragma once
// choose your type of robot
#define BB016 "16"
// #define BB046 "46"
// #define BB011 "11"
```

gripper.h

(/common/constant/gripper.h)

```
/**
 * @name constant of Gripper
 * @author Francisco
 * @date 25-10-2025
*/
#pragma once
#include "common/constant/board.h"
#if defined(BB016) || defined(BB046)
const int GRIPPER = 12;
#endif
#ifdef BB011
const int GRIPPER = 4;
#endif
```

hc12.h

(/common/constant/hc12.h)

```
/**
 * @name constants of HC-12
 * @author Sunny
 * @date 15-12-2025
*/
#pragma once
#include "common/constant/board.h"
#ifdef BB016
const int RX = 11;
const int TX = 13;
#endif
#ifdef BB046
const int RX = 4;
const int TX = 13;
#endif
#ifdef BB011
const int RX = 13;
```

```
const int TX = 1;
#endif
```

motor.h

(/common/constant/motor.h)

```
/**
 * @name constant of motor
 * @authors Fumbre (Vladyslav) & Sunny
 * @date 15-11-2025
*/
#pragma once
const int FULL_PWM_VALUE = 255;
const int FULL_SPEED = 100;                    // In precentage
// 20 pulses per wheel rotation
const int PULSES_PER_ROTATION = 20;
// ======================BB016=======================
#ifdef BB016
const int PIN_MOTOR_RIGHT_FORWARD = 10;
const int PIN_MOTOR_RIGHT_BACKWARD = 6;
const int PIN_MOTOR_LEFT_FORWARD = 5;
const int PIN_MOTOR_LEFT_BACKWARD = 9;
const int PIN_MOTOR_LEFT_PULSE = 2;
const int PIN_MOTOR_RIGHT_PULSE = 3;
// PID factors
const float Kp_front = 4.5;                  // Proportional
const float Ki_front = 2.9;                  // Integral
const float Kd_front = 0.2;                  // Derivative
const float Kp_back = 5.0;
const float Ki_back = 2.2;
const float Kd_back = 0.12;
#endif
// ==================================================
// ======================BB011=======================
#ifdef BB011
const int PIN_MOTOR_RIGHT_FORWARD = 11;
const int PIN_MOTOR_RIGHT_BACKWARD = 10;
const int PIN_MOTOR_LEFT_FORWARD = 9;
const int PIN_MOTOR_LEFT_BACKWARD = 6;
const int PIN_MOTOR_LEFT_PULSE = 2;
const int PIN_MOTOR_RIGHT_PULSE = 3;
// PID factors
const float Kp_front = 4.5; // Proportional
const float Ki_front = 2.9; // Integral
const float Kd_front = 0.2; // Derivative
const float Kp_back = 5.0;
const float Ki_back = 2.2;
const float Kd_back = 0.12;
#endif
// ==================================================
// ======================BB046=======================
#ifdef BB046
const int PIN_MOTOR_RIGHT_FORWARD = 9;
```

```
const int PIN_MOTOR_RIGHT_BACKWARD = 6;
const int PIN_MOTOR_LEFT_FORWARD = 10;
const int PIN_MOTOR_LEFT_BACKWARD = 11;
const int PIN_MOTOR_LEFT_PULSE = 3;
const int PIN_MOTOR_RIGHT_PULSE = 2;
// PID factors
const float Kp_front = 4.0;
const float Ki_front = 1.9;
const float Kd_front = 0.12;
const float Kp_back = 5.9;
const float Ki_back = 2.2;
const float Kd_back = 0.12;
#endif
// ================================================
#if defined(BB046) || defined(BB011) || defined(BB016)
// Pins Motor's Array
const int PINS_MOTOR[] = {
    PIN_MOTOR_RIGHT_FORWARD,
    PIN_MOTOR_RIGHT_BACKWARD,
    PIN_MOTOR_LEFT_FORWARD,
    PIN_MOTOR_LEFT_BACKWARD,
};
// Pins Motor's Array Lenght
const int PINS_MOTOR_LENGTH = sizeof(PINS_MOTOR) / sizeof(int);
#endif
```

neopixel.h

(/common/constant/neopixel.h)

```
/**
 * @name constant of Neopixels
 * @author Sunny
 * @date 15-12-2025
*/
#pragma once
#include "common/constant/board.h"
// ======================BB011======================
#ifdef BB011
// NI = NeoPixel Input
const int PIN_NI = 5;
#endif
// ================================================
// ======================BB016======================
#ifdef BB016
// NI = NeoPixel Input
const int PIN_NI = 4;
#endif
// ================================================
// ======================BB046======================
#ifdef BB046
// NI = NeoPixel Input
const int PIN_NI = 5;
#endif
// ================================================
const int NUMBER_OF_NEOPIXEL = 4;
```

reflective_sensor.h

(/common/constant/reflective_sensor.h)

```
/**
 * @name constant of reflective sensor
 * @author Fumbre (Vladyslav)
 * @date 08-12-2025
*/
#pragma once
#include <Arduino.h>
#include "common/robot/globalvariable/globalVariable.h"
// RS = Reflective Sensor
const int PIN_RS_D8 = A0;
const int PIN_RS_D7 = A1;
const int PIN_RS_D6 = A2;
const int PIN_RS_D5 = A3;
const int PIN_RS_D4 = A4;
const int PIN_RS_D3 = A5;
const int PIN_RS_D2 = A6;
const int PIN_RS_D1 = A7;
// Difference between initial surface and next read
// Example: first value is 255 after 255 + THRESHOLD next surface value
const int THRESHOLD = 275;
// Second RS data margin of error
const int MARGIN_SURFACE = 100;
const int PINS_RS[] = {
    PIN_RS_D8,
    PIN_RS_D7,
    PIN_RS_D6,
    PIN_RS_D5,
    PIN_RS_D4,
    PIN_RS_D3,
    PIN_RS_D2,
    PIN_RS_D1,
};
// Store array length dynamically
const int PINS_RS_LENGTH = sizeof(PINS_RS) / sizeof(int);
```

robot.h

(/common/constant/robot.h)

```
/**
 * @name constants for robot
 * @authors Sunny
 * @date 10-11-2025
*/
#pragma once
#include "board.h"
// Distance between robot's center and wheels' center (cm)
const float ROBOT_RADUIS = 6.5;
// Radius per wheel (cm)
const float WHEEL_RADUIS = 3.6;
// Follow Line
#ifdef BB046
const int SETTING_MODE = 1;
#endif
// Maze Line
```

```
#ifdef BB016
const int SETTING_MODE = 2;
#endif
// Physical Maze
#ifdef BB011
const int SETTING_MODE = 3;
#endif
```

sonar.h

(/common/constant/sonar.h)

```
/**
 * @name constant of Sonar
 * @author Francisco & Uraib
 * @date 11-12-2025
*/
#pragma once
#include "board.h"
#if defined(BB016) || defined(BB046)
const int PIN_SONAR_TRIG = 7;
const int PIN_SONAR_ECHO_FRONT = 8;
#endif
#ifdef BB011
const int PIN_SONAR_TRIG = 12;              // Shared Trigger
const int PIN_SONAR_ECHO_FRONT = 8;         // front sonar echo pin
const int PIN_SONAR_ECHO_RIGHT = 7;         // fleft sonar echo pin
const int PIN_SONAR_ECHO_LEFT = 5;          // right sonar echo pin
#endif
```

/common/robot/

/actions/

(/common/robot/actions/obstacle_avoidance.h)

```
/**
 * @name functions of object avoidance
 * @author Francisco
 * @date 20-11-2025
*/
#include "obstacle_avoidance.h"
bool avoiding = false;
/**
 * @name obstacleAvoidance
 * @author Francisco
 * @date 20-11-2025
 * @details  Executes a non-blocking obstacle avoidance routine using
 * timed movement stages. The robot performs a sequence of turns and
 * short forward movements to bypass an obstacle and then resumes
 * normal operation.
*/
void obstacleAvoidance(int speed) {
  static Timer t;                   // Duration of each movement step
  static int step = 0;              // Stage of the avoidance sequence
  // Start avoidance routine
  if (!avoiding) {
    avoiding = true;                // Mark that avoidance mode has begun
```

```
    step = 0;                       // Reset step sequence to the first movement
    t.resetInterval();              // Reset the timer to ensure timings start fresh
    return;
  }
  switch (step) {
  // Turn left
  case 0:
    moveSpeed(-150, speed);
    if (t.interval(250)) {          // After 250 ms
      step++;
      t.resetInterval();
    }
    break;
  // Move forward
  case 1:
    moveSpeed(speed, speed);
    if (t.interval(150)) {          // After 150 ms
      step++;
      t.resetInterval();
    }
    break;
  // Turn right
  case 2:
    moveSpeed(speed, -150);
    if (t.interval(100)) {          // After 100 ms
      step++;
      t.resetInterval();
    }
    break;
  // Move forward
  case 3:
    moveSpeed(speed, 160);
    if (t.interval(200)) {          // After 200 ms
      step = 4;                     // End of avoidng
      t.resetInterval();
    }
    break;
  case 4:
    avoiding = false;
    break;
  }
}
```

/motor/

(/common/robot/motor/motor.cpp)

```
#include "motor.h"
Timer timestampCountPulsesRight;
Timer timestampCountPulsesLeft;
/**
 * @name countLeftPulses
 * @author Fumbre (Vladyslav)
 * @date 13-11-2025
 * @details count pulses for left motor
*/
void countLeftPulses() {
  if (timestampCountPulsesLeft.executeOnce(0)) {
    motor_left_pulses_counter++;
```

```
  }
  setupPulseCounter();
}
/**
 * @name countRightPulses
 * @author Fumbre (Vladyslav)
 * @date 13-11-2025
 * @details count pulses for right motor
*/
void countRightPulses() {
  if (timestampCountPulsesRight.executeOnce(0)) {
    motor_right_pulses_counter++;
  }
  setupPulseCounter();
}
/**
 * @name setupMotor
 * @author Fumbre (Vladyslav)
 * @date 13-11-2025
 * @details configurate pins for motor
*/
void setupMotor() {
  for (int i = 0; i < PINS_MOTOR_LENGTH; i++) {
    pinMode(PINS_MOTOR[i], OUTPUT);
    digitalWrite(PINS_MOTOR[i], LOW);
  }
  pinMode(PIN_MOTOR_LEFT_PULSE, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_MOTOR_LEFT_PULSE), countLeftPulses, RISING);
  pinMode(PIN_MOTOR_RIGHT_PULSE, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_MOTOR_RIGHT_PULSE), countRightPulses, RISING);
};
void setupPulseCounter() {
  timestampCountPulsesLeft.resetExecuteOnce();
  timestampCountPulsesRight.resetExecuteOnce();
}
```

/movement/

(/common/robot/movement/movement.cpp)

```
#include "movement.h"
static int prevPulsesLeft = 0;
int prevPulsesRigt = 0;
Timer didMoveRightTimer;
Timer didMoveLeftTimer;
/**
 * @name resetMoveLeft
 * @author Fumbre(Vladyslav)
 * @date 27-11-2025
 * @details reset didMoveLeft timer
*/
int resetMoveLeft() {
  didMoveLeftTimer.resetExecuteOnce();
}
/**
 * @name resetMoveRight
 * @author Fumbre(Vladyslav)
 * @date 27-11-2025
 * @details reset didMoveRight timer
```

```
*/
int resetMoveRight() {
  didMoveRightTimer.resetExecuteOnce();
}
/**
 * @name didMoveRight
 * @author Fumbre(Vladyslav)
 * @date 27-11-2025
 * @param speed(0|255)
 * @param pulses(0|999)
 * @return bool
 * @details after pulses rotation return true, otherwise false
*/
bool didMoveRight(int speed, int pulses) {
  if (didMoveRightTimer.executeOnce(0)) {
    prevPulsesLeft = motor_left_pulses_counter;
  }
  if (motor_left_pulses_counter >= prevPulsesLeft + pulses) {
    didMoveRightTimer.resetExecuteOnce();
    moveStopAll();
    return true;
  }
  else {
    moveStabilized(speed, speed * -1);
    return false;
  }
}
/**
 * @name didMoveLeft
 * @author Fumbre(Vladyslav)
 * @date 27-11-2025
 * @param speed(0|255)
 * @param pulses(0|999)
 * @return bool
 * @details after pulses rotation return true, otherwise false
*/
bool didMoveLeft(int speed, int pulses) {
  if (didMoveLeftTimer.executeOnce(0)) {
    prevPulsesRigt = motor_right_pulses_counter;
  }
  if (motor_right_pulses_counter >= prevPulsesRigt + pulses) {
    didMoveLeftTimer.resetExecuteOnce();
    moveStopAll();
    return true;
  }
  else {
    moveStabilized(speed * -1, speed);
    return false;
  }
}
/**
 * @name moveStop
 * @author Fumbre(Vladyslav)
 * @date 13-11-2025
 * @details stop specific motor pin
*/
void moveStop(int motor_pin) {
  digitalWrite(motor_pin, LOW);
};
/**
```

```
 * @name moveStopAll
 * @author Fumbre(Vladyslav)
 * @date 13-11-2025
 * @details stop all motor pins
*/
void moveStopAll() {
  for (int i = 0; i < PINS_MOTOR_LENGTH; i++) {
    digitalWrite(PINS_MOTOR[i], LOW);
  }
};
/**
 * @name moveSpeed
 * @author Fumbre(Vladyslav)
 * @date 26-11-2025
 * @param speedLeft(-255|255)
 * @param speedRight(-255|255)
 * @details this function doesn't count your pulses
 * @details you can use speed from -255 to 255
*/
void moveSpeed(int speedLeft, int speedRight) {
  if (speedLeft < 0) {
    analogWrite(PIN_MOTOR_LEFT_FORWARD, LOW);
    analogWrite(PIN_MOTOR_LEFT_BACKWARD, speedLeft * -1);
  }
  else {
    analogWrite(PIN_MOTOR_LEFT_BACKWARD, LOW);
    analogWrite(PIN_MOTOR_LEFT_FORWARD, speedLeft);
  }
  if (speedRight < 0) {
    analogWrite(PIN_MOTOR_RIGHT_FORWARD, LOW);
    analogWrite(PIN_MOTOR_RIGHT_BACKWARD, speedRight * -1);
  }
  else {
    analogWrite(PIN_MOTOR_RIGHT_BACKWARD, LOW);
    analogWrite(PIN_MOTOR_RIGHT_FORWARD, speedRight);
  }
}
/**
 * @name moveStabilized
 * @author Fumbre(Vladyslav)
 * @date 26-11-2025
 * @param speedLeft(-255|255)
 * @param speedRight(-255|255)
 * @details movement is stabilized by pulses
 * @details you can use speed from -255 to 255
*/
void moveStabilized(int speedLeft, int speedRight) {
  #define LEFT_CORRECTION 1.00
  #define RIGHT_CORRECTION 0.93
  speedLeft = speedLeft * LEFT_CORRECTION;
  speedRight = speedRight * RIGHT_CORRECTION;
  static int step = 0;
  static Timer stampForward;
  static Timer stampRotateLeft;
  static Timer stampRotateRight;
  switch (step) {
    case 0:
      if (stampForward.interval(20)) {
        writeSpeed(speedLeft, speedRight);
      }
```

```
      if (stampForward.executeOnce(40)) {
        step++;
      }
      break;
    case 1:
      if (motor_left_pulses_counter < motor_right_pulses_counter) {
        if (stampRotateLeft.interval(0)) {
          writeSpeed(speedLeft, speedRight / 1.8);
        }
        if (stampRotateLeft.executeOnce(20)) {
          step++;
        }
      }
      else {
        step++;
      }
      break;
    case 2:
      if (motor_left_pulses_counter > motor_right_pulses_counter) {
        if (stampRotateRight.interval(0)) {
          writeSpeed(speedLeft / 1.8, speedRight);
        }
        step++;
      }
      else {
        step++;
      }
      break;
    case 3:
      stampForward.hardReset();
      stampRotateLeft.hardReset();
      stampRotateRight.hardReset();
      step = 0;
      break;
  }
}
/**
 * @name writeSpeed
 * @author Fumbre(Vladyslav)
 * @date 26-11-2025
 * @param speedLeft(-255|255)
 * @param speedRight(-255|255)
 * @details put to left and right motors high depending on values
 * @details for minus value motor go backward
 * @details this funciton put all pins to LOW (used for moveStabilized)
*/
void writeSpeed(int speedLeft, int speedRight) {
  for (int i = 0; i < PINS_MOTOR_LENGTH; i++) {
    analogWrite(PINS_MOTOR[i], LOW);
  }
  if (speedLeft < 0) {
    analogWrite(PIN_MOTOR_LEFT_BACKWARD, speedLeft * -1);
  }
  else {
    analogWrite(PIN_MOTOR_LEFT_FORWARD, speedLeft);
  }
  if (speedRight < 0) {
    analogWrite(PIN_MOTOR_RIGHT_BACKWARD, speedRight * -1);
  }
  else {
```

```
    analogWrite(PIN_MOTOR_RIGHT_FORWARD, speedRight);
  }
}
```

/sonar/

(/common/robot/sonar/sonar.cpp)

```
#include "sonar.h"
/**
 * @name setupSonar
 * @authors Francisco
 * @date 11-12-2025
 * @details Initializes sonar pins depending on the robot model.
 * @details For BB011: uses shared trigger and three echo pins (front, right, left).
 * @details For other robots: uses one trigger and one front echo.
*/
void setupSonar() {
#if defined(BB011)
    pinMode(PIN_SONAR_TRIG, OUTPUT);
    pinMode(PIN_SONAR_ECHO_FRONT, INPUT);
    pinMode(PIN_SONAR_ECHO_RIGHT, INPUT);
    pinMode(PIN_SONAR_ECHO_LEFT, INPUT);
#else
    pinMode(PIN_SONAR_TRIG, OUTPUT);
    pinMode(PIN_SONAR_ECHO_FRONT, INPUT);
#endif
}
/**
 * @name measureDistance
 * @authors Francisco
 * @date 11-12-2025
 * @param echo Pin number of the sonar echo
 * @details Triggers the sonar pulse and measures the echo duration.
 * @details Uses a 15 ms timeout (~250 cm max range) for faster measurements.
 * @return float Distance in centimeters; returns 400.0 if no echo is detected.
*/
float measureDistance(int echo) {
#if defined(BB011)
    delay(35);
#endif
    digitalWrite(PIN_SONAR_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(PIN_SONAR_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_SONAR_TRIG, LOW);
    unsigned long duration = pulseIn(echo, HIGH, 15000);
    if (duration == 0) return 400.0;
    return duration * 0.0343 / 2;
}
/**
 * @name getDistanceCM_Front
 * @authors Francisco
 * @date 11-12-2025
 * @details Measures the distance from the front sonar
 * and stores it in sonarSendDatafrontDistance.
 * @return float Distance in centimeters
*/
```

```
float getDistanceCM_Front() {
    float distance = measureDistance(PIN_SONAR_ECHO_FRONT);
    sonarSendDatafrontDistance = distance;
    return distance;
}
/**
 * @name getDistanceCM_Right
 * @authors Francisco
 * @date 11-12-2025
 * @details Measures the distance from the right sonar (BB011 only)
 * and stores it in sonarSendDataRightDistance.
 * @return float Distance in centimeters; returns 400.0 for robots without right sonar
*/
float getDistanceCM_Right() {
#if defined(BB011)
    float distance = measureDistance(PIN_SONAR_ECHO_RIGHT);
    sonarSendDataRightDistance = distance;
    return distance;
#else
    return 400.0;
#endif
}
/**
 * @name getDistanceCM_Left
 * @authors Francisco
 * @date 11-12-2025
 * @details Measures the distance from the left sonar (BB011 only)
 * and stores it in sonarSendDataLeftDistance.
 * @return float Distance in centimeters; returns 400.0 for robots without left sonar
*/
float getDistanceCM_Left() {
#if defined(BB011)
    float distance = measureDistance(PIN_SONAR_ECHO_LEFT);
    sonarSendDataLeftDistance = distance;
    return distance;
#else
    return 400.0;
#endif
}
/**
 * @name isObstacleFront
 * @authors Francisco
 * @date 11-12-2025
 * @param limit Maximum distance to consider as obstacle (cm)
 * @details Checks if there is an obstacle in front within a certain distance range.
 * @return bool True if an obstacle is detected, otherwise false
*/
bool isObstacleFront(float limit) {
    float d = getDistanceCM_Front();
    return (d > 2.0 && d <= limit);
}
/**
 * @name isObstacleRight
 * @authors Francisco
 * @date 11-12-2025
 * @param limit Maximum distance to consider as obstacle (cm)
 * @details Checks if there is an obstacle to the right within a certain distance range.
 * @return bool True if an obstacle is detected, otherwise false
*/
bool isObstacleRight(float limit) {
```

```
    float d = getDistanceCM_Right();
    return (d > 2.0 && d <= limit);
}
/**
 * @name isObstacleLeft
 * @authors Francisco
 * @date 11-12-2025
 * @param limit Maximum distance to consider as obstacle (cm)
 * @details Checks if there is an obstacle to the left within a certain distance range.
 * @return bool True if an obstacle is detected, otherwise false
*/
bool isObstacleLeft(float limit) {
    float d = getDistanceCM_Left();
    return (d > 2.0 && d <= limit);
}
```

/reflective_sensor/

(/common/robot/reflective_sensor/reflecctive_sensor.h)

```
/**
 * @name ReflectiveSensor
 * @authors Fumbre (Vladyslav) & Aria & Sunny & Uraib
 * @date 08-12-2025
*/
#pragma once
#include <Arduino.h>
#include "common/tools/ARRAY_SIZE.h"
#include "common/constant/reflective_sensor.h"
#include "common/tools/Timer.h"
// Bitmask tables
const uint8_t centerPatterns[] = {
    0b00011000,
    0b00111100};
const uint8_t slightLeftPatterns[] = {
    0b01100000,
    0b00110000,
    0b01110000,
    0b0011100,
};
const uint8_t slightRightPatterns[] = {
    0b00000110,
    0b00001100,
    0b00001110,
    0b00011100,
};
const uint8_t hardLeftPatterns[] = {
    0b11100000,
    0b11000000,
    0b10000000,
};
const uint8_t hardRightPatterns[] = {
    0b00000111,
    0b00000011,
    0b00000001,
};
const uint8_t allWhite[] = {0b00000000};
const uint8_t allBlack[] = {0b11111111};
const uint8_t leftTurn[] = {0b11111000, 0b11110000, 0b11111100};
```

```
const uint8_t rightTurn[] = {0b00011111, 0b00001111, 0b00111111};
enum LineState {
    CENTER,
    SLIGHT_LEFT,
    SLIGHT_RIGHT,
    HARD_LEFT,
    HARD_RIGHT,
    ALL_WHITE,
    ALL_BLACK,
    LEFT_TURN,
    RIGHT_TURN,
    OTHER
};
/**
 * @name ReflectiveRead
 * @authors Uraib
 * @date 04-12-2025
 * @details Structure to maintain running statistics for reflective sensor readings.
 * @details Tracks the number of readings, the mean, variance (using Welford's algorithm),
 * and the minimum and maximum values observed.
 * @details The update() method incorporates a new sensor reading into the statistics effi
ciently,
 * updating mean, variance, minimum, and maximum without storing all previous values.
*/
struct ReflectiveRead {
    unsigned long count = 0;                // Number of readings processed
    double mean = 0.0;                      // Current mean of the readings
    double m2 = 0.0;                        // Sum of squares of differences from the curr
ent mean
    int minimum = 1023;                     // Minimum value observed
    int maximum = 0;                        // Maximum value observed
    /**
     * @name update
     * @param x New sensor reading to incorporate
     * @details Updates the running statistics (mean, variance, min, max) with the new val
ue x.
    */
    void update(int x) {
        count = 1;
        double dx = x - mean;
        mean += dx / count;
        double dx2 = x - mean;
        m2 += dx * dx2;
        if (x < minimum) {
            minimum = x;
        }
        if (x > maximum) {
            maximum = x;
        }
    }
};
/**
 * @name ReflectiveSensor
 * @authors Sunny & Vlad
 * @date 08-12-2025
 * @details Class to manage reflective sensors for line detection.
 * @details Handles calibration, reading of surfaces, and comparison to predefined pattern
s.
 * @details Supports two-surface detection (first and second surface)
 * and provides bitmask status for black line detection.
```

```
*/
class ReflectiveSensor {
private:
    const int *pins;                                    // Array of reflective sensor pins
    int pins_rs_length;                                 // Number of reflective sensor pins
    float threshold;                                    // Threshold for detecting second surf
ace
    int marginError;                                    // Margin to ignore small differences
in surface detection
    ReflectiveRead *reflectiveReadInit;                 // Initial surface data
    ReflectiveRead reflectiveReadBlack[8];              // Calibrated second surface data
    uint8_t blackCalibration;                           // Bitmask of calibrated sensors for b
lack surface
    int blackCalibratedLength = 0;                      // Number of sensors calibrated for bl
ack
    bool isBlackCalibrated = false;                     // True if black calibration completed
    ReflectiveRead *currentSensors;                     // Current sensor readings
    /**
     * @name getRSValue
     * @authors Fumbre
     * @date 04-12-2025
     * @details Reads analog values from reflective sensors and updates running statistics
.
     * @return Pointer to an array of 8 ReflectiveRead objects containing current statisti
cs.
    */
    ReflectiveRead *getRSValue() {
        ReflectiveRead *stats = new ReflectiveRead[8];
        for (int i = 0; i < PINS_RS_LENGTH; ++i) {
            int v = analogRead(PINS_RS[i]);
            *RS_SEND_DATA_RAW_ARRAY[i] = v;
            stats[i].update(v);
        }
        return stats;
    }
    /**
     * @name calibrationInit
     * @authors Sunny & Vlad
     * @date 10-12-2025
     * @details Performs initial calibration for the first surface
     * and stores it in reflectiveReadInit.
    */
    void calibrationInit() {
        free(this->reflectiveReadInit);
        this->reflectiveReadInit = getRSValue();
    }
    /**
     * @name calibrationBlack
     * @authors Fumbre
     * @date 10-12-2025
     * @details Performs calibration for the black surface, updating reflectiveReadBlack a
nd calibration status.
    */
    void calibrationBlack() {
        if (this->blackCalibratedLength < this->pins_rs_length) {
            uint8_t reflectiveReadChange = this->getLineDifference(this-
>reflectiveReadInit, this->threshold);
            for (int i = 0; i < pins_rs_length; i++) {
                if (!(reflectiveReadChange & (128 >> i)) && !(this-
>blackCalibration & (128 >> i))) {
```

```
                    this->reflectiveReadBlack[i].mean = analogRead(PINS_RS[i]);
                    this->blackCalibration |= (128 >> i);
                    this->blackCalibratedLength++;
                }
            }
        } else {
            this->isBlackCalibrated = true;
        }
    }
    /**
     * @name getLineStatusMoreThan
     * @authors Fumbre & Sunny
     * @date 04-12-2025
     * @param compare Pointer to array of ReflectiveRead objects to compare against
     * @param reflectiveDifferenceMargin Margin to ignore small differences
     * @details Compares current sensor readings to 'compare' and returns bitmask status.
     * @return uint8_t Bitmask where 1 indicates current reading >= compare + margin.
    */
    uint8_t getLineStatusMoreThan(ReflectiveRead *compare, int reflectiveDifferenceMargin)
 {
        uint8_t status = 0;
        free(this->currentSensors);
        this->currentSensors = getRSValue();

        for (int i = 0; i < pins_rs_length; i++) {
            if (currentSensors[i].mean + reflectiveDifferenceMargin >= compare[i].mean) {
                status |= (128 >> i);
            } else {
                status &= ~(128 >> i);
            }
        }
        return status;
    }
    /**
     * @name getLineDifference
     * @authors Fumbre & Sunny
     * @date 10-12-2025
     * @param compare Pointer to array of ReflectiveRead objects to compare against
     * @param reflectiveDifference Allowed difference to consider readings equal
     * @details Compares current sensor readings to 'compare' within a tolerance and retur
ns a bitmask.
     * @return uint8_t Bitmask where 1 indicates current reading within difference range o
f compare.
    */
    uint8_t getLineDifference(ReflectiveRead *compare, int reflectiveDifference) {
        uint8_t status = 0;
        free(this->currentSensors);
        this->currentSensors = getRSValue();

        for (int i = 0; i < pins_rs_length; i++) {
            if ((currentSensors[i].mean - reflectiveDifference <= compare[i].mean) &&
                (currentSensors[i].mean + reflectiveDifference >= compare[i].mean)) {
                status |= (128 >> i);
            } else {
                status &= ~(128 >> i);
            }
        }
        return status;
    }
public:
```

```
    /**
     * @name ReflectiveSensor
     * @authors Sunny & Vlad
     * @date 08-12-2025
     * @details Constructor for ReflectiveSensor class.
     * @param pins Array of sensor pins
     * @param pins_rs_length Length of pins array
     * @param threshold Threshold for second surface detection
     * @param marginError Margin to ignore minor differences in detection
    */
    ReflectiveSensor(const int *pins, const int pins_rs_length, const float threshold, con
st int marginError) {
        this->pins = pins;
        this->pins_rs_length = pins_rs_length;
        this->threshold = threshold;
        this->marginError = marginError;
    }
    /**
     * @name setup
     * @authors Sunny & Vlad
     * @date 08-12-2025
     * @details Initializes the reflective sensor pins as INPUT.
    */
    void setup() {
        for (int i = 0; i < pins_rs_length; i++) {
            pinMode(pins[i], INPUT);
        }
    }
    /**
     * @name readBlackLine
     * @authors Fumbre
     * @date 08-12-2025
     * @details Reads the black line using the calibrated second surface.
     * @details If black surface is not fully calibrated, returns 0. Updates RSSendDataSta
tus.
     * @return uint8_t Bitmask representing current black line status.
    */
    uint8_t readBlackLine() {
        static Timer t;
        if (t.executeOnce(0)) {
            this->calibrationInit();
        }
        this->calibrationBlack();
        if (!this->isBlackCalibrated) {
            return 0;
        }
        uint8_t currentBlackStatus = this->getLineStatusMoreThan(this-
>reflectiveReadBlack, this->marginError);
        String statusStr = "";
        for (int i = 7; i >= 0; i--) {
            statusStr += (currentBlackStatus & (1 << i)) ? '1' : '0';
        }
        RSSendDataStatus = statusStr;
        return currentBlackStatus;
    }
    /**
     * @name match
     * @authors Aria & Fumbre
     * @date 12-12-2025
     * @param patterns Array of patterns to match
```

```
     * @param elementCount Length of the patterns array
     * @details Compares current black line status against a list of patterns.
     * @return bool True if any pattern matches, otherwise false
    */
    bool match(const uint8_t *patterns, int elementCount) {
        for (int i = 0; i < elementCount; i++) {
            if (this->readBlackLine() == patterns[i]) {
                return true;
            }
        }
        return false;
    }
    /**
     * @name pattern
     * @authors Aria
     * @date 12-12-2025
     * @details Determines the line state based on current black line pattern.
     * @return LineState Enum indicating position (CENTER, SLIGHT_LEFT, SLIGHT_RIGHT, etc.
)
    */
    LineState pattern() {
        if (match(centerPatterns, ARRAY_SIZE(centerPatterns))) return CENTER;
        if (match(slightLeftPatterns, ARRAY_SIZE(slightLeftPatterns))) return SLIGHT_LEFT;
        if (match(slightRightPatterns, ARRAY_SIZE(slightRightPatterns))) return SLIGHT_RIG
HT;
        if (match(hardLeftPatterns, ARRAY_SIZE(hardLeftPatterns))) return HARD_LEFT;
        if (match(hardRightPatterns, ARRAY_SIZE(hardRightPatterns))) return HARD_RIGHT;
        if (match(leftTurn, ARRAY_SIZE(leftTurn))) return LEFT_TURN;
        if (match(rightTurn, ARRAY_SIZE(rightTurn))) return RIGHT_TURN;
        if (match(allWhite, ARRAY_SIZE(allWhite))) return ALL_WHITE;
        if (match(allBlack, ARRAY_SIZE(allBlack))) return ALL_BLACK;
        return OTHER;
    }
};
```

/gripper/

(/common/robot/gripper/gripper.cpp)

```
#include "gripper.h"
/**
 * @name gripper
 * @authors Francisco
 * @date 3-12-2025
 * @details Configures the gripper control pin and ensures
 * the servo signal starts in a safe LOW state.
*/
void setupGripper() {
    pinMode(GRIPPER, OUTPUT);
    digitalWrite(GRIPPER, LOW);
}
/**
 * @name gripper
 * @authors Francisco
 * @date 3-12-2025
 * @details Generates non-blocking servo control pulses in software to control a gripper.
 * @details Maintains a 20 ms interval between pulses, ensuring correct servo operation.
 * @param pulseMicroseconds Pulse width in microseconds that sets the gripper position.
 *  - Typical range for standard servos: 1000 µs (fully closed) to 2000 µs (fully open).
```

```
 *  - If pulseMicroseconds <= 0, the last set pulse is maintained.
 * @note This function should be called repeatedly in the main loop to continuously contro
l the servo.
*/
void gripper(int pulseMicroseconds) {
    static unsigned long nextPulseTime = 0;
    static int lastPulse = 2000;
    if (pulseMicroseconds > 0) {
        lastPulse = pulseMicroseconds;
    }
    unsigned long now = millis();
    if (now >= nextPulseTime) {
        nextPulseTime = now + 20;
        digitalWrite(GRIPPER, HIGH);
        delayMicroseconds(lastPulse);
        digitalWrite(GRIPPER, LOW);
    }
}
/**
 * @name gripperCatch
 * @authors Francisco
 * @date 3-12-2025
 * @details Closes the gripper.
*/
void gripperCatch() {
    gripper(1000);
    gripperStatus = true;
}
void gripperUnCatch() {
    gripper(2000);
    gripperStatus = false;
}
```

/neopixel/

(/common/robot/neopixel/neopixel.cpp)

```
#include "neopixel.h"
Adafruit_NeoPixel strip(NUMBER_OF_NEOPIXEL, PIN_NI, NEO_RGB + NEO_KHZ800);
/**
 * @name initNeopixelPins
 * @author Sunny
 * @date 24-11-2025
*/
void initNeopixelPins() {
  strip.begin();
  strip.clear();
}
/**
 * @name initNeopixelPins
 * @author Sunny
 * @date 26-11-2025
 * @param R red color value (0-255)
 * @param G green color value (0-255)
 * @param B blue color value (0-255)
*/
void turnOnAllLeds(int R, int G, int B) {
  for (int i = 0; i < 4; i++) {
    strip.setPixelColor(i, strip.Color(R, G, B));
```

```
    colorArray[i][0] = R;
    colorArray[i][1] = G;
    colorArray[i][2] = B;
  }
  strip.show();
}
/**
 * @name turnOffAllLeds
 * @author Sunny
 * @date 26-11-2025
*/
void turnOffAllLeds() {

  for (int i = 0; i < 4; i++) {
    strip.setPixelColor(i, strip.Color(0, 0, 0));
    colorArray[i][0] = 0;
    colorArray[i][1] = 0;
    colorArray[i][2] = 0;
  }
  strip.show();
}
/**
 * @name turnOnSomeLeds
 * @author Sunny
 * @date 26-11-2025
 * @param index array of led index
 * @param length length of led index array
 * @param R red color value (0-255)
 * @param G green color value (0-255)
 * @param B blue color value (0-255)
*/
void turnOnSomeLeds(int index[], int length, int R, int G, int B) {
  for (int i = 0; i < length; i++) {
    strip.setPixelColor(index[i], strip.Color(R, G, B));
    colorArray[index[i]][0] = R;
    colorArray[index[i]][1] = G;
    colorArray[index[i]][2] = B;
  }
  strip.show();
}
/**
 * @name turnOffSomeLeds
 * @author Sunny
 * @date 26-11-2025
 * @param index array of led index
 * @param length length of led index array
*/
void turnOffSomeLeds(int index[], int length) {
  for (int i = 0; i < length; i++) {
    colorArray[index[i]][0] = 0;
    colorArray[index[i]][1] = 0;
    colorArray[index[i]][2] = 0;
    strip.setPixelColor(index[i], strip.Color(0, 0, 0));
  }
  strip.show();
}
void turnOnOneLed(int index, int R, int G, int B) {
  strip.setPixelColor(index, strip.Color(R, G, B));
  strip.show();
}
```

/hc12/

(/common/robot/hc12/hc12.cpp)

```
#include "hc12.h"
// Create a SoftwareSerial object to communicate with the HC-12 module
SoftwareSerial hc12(RX, TX);
/**
 * @name buildHC12Connection
 * @author Sunny
 * @date 15-12-2025
 * @details build connection between HC-12 and Arduino
*/
void buildHC12Connection() {
    hc12.begin(9600);
}
/**
 * @name sendDataFromHC12
 * @author Sunny
 * @date 15-12-2025
 * @details send data through HC-12
 * @param data the data that needs send
*/
void sendDataFromHC12(String data) {
    if (!data)
        return;
    hc12.println(data);
}
/**
 * @name receiveDataFromHC12
 * @author Sunny
 * @date 23-12-2025
 * @details Reads data from the HC-12 wireless module.
 * @details Checks if data is available, and if so, reads a complete line terminated by '\
n'.
 * @details Returns an empty string if no data is available.
 * @return String containing the received data. If no data is received, returns an empty s
tring.
*/
String receiveDataFromHC12() {
    if (hc12.available() > 0) {
        return hc12.readStringUntil('\n');
    }
    return "";
}
```

/datasend/

(/common/robot/datasend/datasend.cpp)

```
#include "datasend.h"
int interval = 2000;
String type = "o";
Timer timer;
Timer timer2;
/**
 * @name dataSend
```

```
 * @author Sunny
 * @date 10-12-2025
 * @details Calls all the data sending functions sequentially:
 *  - sendGripperData()
 *  - sendNeopixelData()
 *  - sendReflectiveSensorData()
 *  - sendSonarData()
*/
void dataSend() {
    sendGripperData();
    sendNeopixelData();
    sendReflectiveSensorData();
    sendSonarData();
}
/**
 * @name sendSonarData
 * @author Sunny
 * @date 10-12-2025
 * @details Sends sonar sensor data (front, right, left) via HC12 every 'interval' millise
conds.
 * @details Builds a message: "o,so,<robotCode>,0,<frontDistance>,1,<rightDistance>,2,<lef
tDistance>"
 * @details Resets the timer after sending.
*/
void sendSonarData() {
    if (timer.interval(interval)) {
        String robotCode = getCurrentRobotCode();
        String msg = "o,so,";
        msg += (robotCode + ",");
        msg += "0,";
        msg += sonarSendDatafrontDistance;
        msg += ",1,";
        msg += sonarSendDataRightDistance;
        msg += ",2,";
        msg += sonarSendDataLeftDistance;
        sendDataFromHC12(msg);
        timer.resetInterval();
    }
}
/**
 * @name sendGripperData
 * @author Sunny
 * @date 10-12-2025
 * @details Sends the gripper status via HC12 every 'interval' milliseconds.
 * @details Builds a message: "o,g,<robotCode>,<gripperStatus>"
 * @details gripperStatus is 1 if true (closed), 0 if false (open).
*/
void sendGripperData() {
    if (timer2.interval(interval)) {
        String robotCode = getCurrentRobotCode();
        String msg = "o,g,";
        msg += (robotCode + ",");
        msg += (gripperStatus ? "1" : "0");
        sendDataFromHC12(msg);
    }
}
/**
 * @name sendReflectiveSensorData
 * @author Sunny
 * @date 10-12-2025
```

```
 * @details Sends reflective sensor data once every 'interval' milliseconds using executeO
nce().
 * @details Builds a message: "o,rs,<robotCode>,<sensor0>,...,<sensor7>,<status>"
 * @details Resets executeOnce timer after sending.
*/
void sendReflectiveSensorData()
{
    if (timer.executeOnce(interval)) {
        String msg = "o,rs,";
        msg += getCurrentRobotCode();
        for (int i = 0; i < 8; i++) {
            msg += ",";
            msg += *RS_SEND_DATA_RAW_ARRAY[i];
        }
        msg += ",";
        msg += RSSendDataStatus;
        sendDataFromHC12(msg);
        timer.resetExecuteOnce();
    }
}
/**
 * @name sendNeopixelData
 * @author Sunny
 * @date 10-12-2025
 * @details Sends Neopixel LED color data every 'interval' milliseconds via HC12.
 * @details Builds a message: "o,n,<robotCode>,<led0,r,g,b>,...,<led3,r,g,b>"
 * @details Resets the timeout timer after sending.
*/
void sendNeopixelData() {
    if (timer.timeout(interval)) {
        String robotCode = getCurrentRobotCode();
        String msg = "o,n,";
        msg += (robotCode + ",");
        for (int i = 0; i < 4; i++) {
            msg += i;
            msg += ",";
            for (int j = 0; j < 3; j++) {
                msg += colorArray[i][j];
                if (i != 3 || j != 2) {
                    msg += ",";
                }
            }
        }
        sendDataFromHC12(msg);
        timer.resetTimeout();
    }
}
/**
 * @name getCurrentRobotCode
 * @author Sunny
 * @date 10-12-2025
 * @details Returns the current robot code depending on preprocessor definitions:
 *  - BB046, BB011, or BB016
 * @return String with the robot code
*/
String getCurrentRobotCode() {
    String robotCode = "";
#ifdef BB046
    robotCode = BB046;
#endif
```

```
#ifdef BB011
    robotCode = BB011;
#endif
#ifdef BB016
    robotCode = BB016;
#endif
    return robotCode;
}
```

/globalvariable/

(/common/robots/globalvariable/globalVariable.cpp)

```
#include "globalVariable.h"
// ============================== motor pulses count ==============================
volatile long motor_left_pulses_counter = 0;
volatile long motor_right_pulses_counter = 0;
// ================================================================================
// ================================== gripper =====================================
bool gripperStatus = false;
// ================================================================================
// ================================== neopixels ===================================
int colorArray[4][3] = {{0, 0, 0},
                        {0, 0, 0},
                        {0, 0, 0},
                        {0, 0, 0}};

// ================================================================================
// ============================== reflective sensor ===============================
int a0 = 0;
int a1 = 0;
int a2 = 0;
int a3 = 0;
int a4 = 0;
int a5 = 0;
int a6 = 0;
int a7 = 0;
int *RS_SEND_DATA_RAW_ARRAY[8] = {&a0, &a1, &a2, &a3, &a4, &a5, &a6, &a7};
String RSSendDataStatus = "00000000";
//================================================================================
// ================================== sonar =======================================
float sonarSendDatafrontDistance = 0.0;
float sonarSendDataLeftDistance = 0.0;
float sonarSendDataRightDistance = 0.0;
```

/common/tools/

ARRAY_SIZE.h

(/common/tools/ARRAY_SIZE.h)

```
#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))
```

PID.h

(/common/tools/PID.h)

```
/**
 * @name PID class
 * @author Sunny
 * @date 27-11-2025
*/
class PID {
private:
  float kp;                                 // Proportional factor
  float ki;                                 // Integral factor
  float kd;                                 // Derivative factor
  float lastError = 0;                      // Error in last time
```

```
public:
  float integral = 0;                       // record total error

  /**
   * @name PID
   * @name Sunny
   * @date 27-11-2025
   * @details constructure functions of PID
   * @param kp Proportional factor
   * @param ki Integral factor
   * @param kd Derivative factor
  */

  PID(float kp, float ki, float kd) {
    this->kp = kp;
    this->ki = ki;
    this->kd = kd;
  }
  /**
   * @name caculateCorrection
   * @author Sunny
   * @date 27-11-2025
   * @param v1 first value
   * @param v2 second value
   * @return correction
  */

  float caculateCorrection(float v1, float v2) {

    // Caculate error
    float error = v1 - v2;
    // Plus error to integeral
    integral += error;
    // Caculate derivative
    float derivative = (error - lastError);
    lastError = error;
    // Caculate correction
    float correction = kp * error + ki * integral + kd * derivative;
    return correction;
  }
};
```

Timer.h

(/common/tools/Timer.h)

```
#pragma once
#include <Arduino.h>
/**
 * @name Timer
 * @author Fumbre (Vladyslav)
 * @date 14-11-2025
 * @details Timer class for easier access to millis calculation
*/
class Timer {
private:

  // Timestamp for timer methods
  unsigned long timestampInterval = 0;
```

```
  unsigned long timestampIntervalStart = 0;
  unsigned long timestampExecuteOnce = 0;
  bool isTriggeredExecuteOnce = false;
  unsigned long timestampTimeout = 0;
public:
  /**
   * @name interval
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @param milliseconds how much time should pass to return true
   * @param executionTime how long it returns true
   * @details Timer.interval(500) returns True every 500 milliseconds, otherwise False
   * @details Timer.interval(200, 500) wait 200 milliseconds and return True for 500 milli
seconds
   * @details Timer.interval(0, 500) WRONG!!! RETURNS FOREVER TRUE
   * @return bool
  */

  bool interval(unsigned long milliseconds, unsigned long executionTime = 0) {

    if (timestampInterval == 0) {
      timestampInterval = millis();
    }
    unsigned long now = millis();

    if (now - timestampInterval >= milliseconds) {

      if (now - timestampInterval >= milliseconds + executionTime) {
        timestampInterval = now;
      }
      return true;
    }
    return false;
  }
  /**
   * @name intervalStart
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @param milliseconds how much time should pass to return true
   * @param executionTime how long it returns true
   * @details When executed for first time returns True immidiately
   * @details Timer.intervalStart(500) returns True every 500 milliseconds, otherwise Fals
e
   * @details Timer.intervalStart(200, 500) wait 200 milliseconds and return True for 500
milliseconds
   * @details Timer.intervalStart(0, 500) WRONG!!! RETURNS FOREVER TRUE
   * @return bool
  */
  bool intervalStart(unsigned long milliseconds, unsigned long executionTime = 0) {
    if (timestampIntervalStart == 0) {
      timestampIntervalStart = millis();
      return true;
    }
    unsigned long now = millis();
    if (now - timestampIntervalStart >= milliseconds) {
      if (now - timestampIntervalStart >= milliseconds + executionTime) {
        timestampIntervalStart = now;
      }
      return true;
    }
```

```
    return false;
  }
  /**
   * @name executeOnce
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @param milliseconds returns true after milliseconds only once
   * @details Timer.executeOnce(500) will return True once and only once! even in a loop i
t will be executed once after n milliseconds!!
   * @return bool
  */
  bool executeOnce(unsigned long milliseconds, unsigned long executedTime = 0) {

    if (isTriggeredExecuteOnce) {
      return false;
    }
    unsigned long now = millis();
    if (timestampExecuteOnce == 0) {
      timestampExecuteOnce = now;
    }
    if (now - timestampExecuteOnce >= milliseconds) {
      if (now - timestampExecuteOnce >= milliseconds + executedTime) {
        isTriggeredExecuteOnce = true;
      }
      return true;
    }
    return false;
  }
  /**
   * @name timeout
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @param milliseconds use time in milliseconds
   * @details Timer.timeout(300) returns true forever after 300 milliseconds
   * @return bool
  */
  bool timeout(unsigned long shouldPass) {
    if (timestampTimeout == 0) {
      timestampTimeout = millis();
    }
    unsigned long now = millis();
    if (now - timestampTimeout >= shouldPass) {
      return true;
    }
    return false;
  }
  /**
   * @name hardReset
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @details RESET ALL TIMERS
  */
  void hardReset() {
    timestampInterval = 0;
    timestampExecuteOnce = 0;
    timestampTimeout = 0;
    isTriggeredExecuteOnce = false;
  }
  /**
   * @name resetInterval
```

```
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @details reset interval timer
  */
  void resetInterval() {
    timestampInterval = 0;
  }
  /**
   * @name resetIntervalStart
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @details reset intervalStart timer
  */
  void resetIntervalStart() {
    timestampIntervalStart = 0;
  }
  /**
   * @name resetExecuteOnce
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @details reset executeOnce timer
  */
  void resetExecuteOnce() {
    timestampExecuteOnce = 0;
    isTriggeredExecuteOnce = false;
  }
  /**
   * @name resetTimeout
   * @author Fumbre (Vladyslav)
   * @date 14-11-2025
   * @details reset timeout timer
  */
  void resetTimeout() {
    timestampTimeout = 0;
  }
};
```

Bluetooth.h

(/common/tools/bluetooth.h)

```
#pragma once
#include <SoftwareSerial.h>
// Init bluetooth pins
SoftwareSerial blueTooth(1, 0);
```

Neopixel.h

(/common/tools/Neopixel.h)

```
/**
 * @name Neopixel
 * @author Sunny
 * @date 15-12-2025
 * @details Class to control WS2812/Neopixel RGB LEDs.
 * @details Manages LED colors using RGB structs
 * and updates LEDs via direct port manipulation for speed.
```

```
 * @details Supports setting individual LEDs, filling ranges, clearing, and updating color
s.
*/
#pragma once
#include <Arduino.h>
#define MAX_COLOUR_VALUE 255
struct RGB {
    uint8_t R;
    uint8_t G;
    uint8_t B;
};
class Neopixel {
private:
    int PIN_NI;                     // Data pin for LED strip
    int NUMBER_OF_LED;              // Total number of LEDs
    RGB *color;                     // Current RGB color array
    RGB *lastColor;                 // Previous RGB color array
    volatile uint8_t *port;         // PORT register pointer for fast pin writes
    uint8_t bitMask;                // Pin mask for PORT manipulation
    /**
     * @name initinfo
     * @author Sunny
     * @date 26-11-2025
     * @details Determines the correct PORT and bit mask for the data pin.
    */

    void initinfo() {
        uint8_t pin = PIN_NI;
        if (pin <= 7) {
            port = &PORTD;
            bitMask = 1 << pin;
        } else if (pin <= 13) {
            port = &PORTB;
            bitMask = 1 << (pin - 8);
        } else if (pin >= 14 && pin <= 19) {
            port = &PORTC;
            bitMask = 1 << (pin - 14);
        }
    }
    /**
     * @name checkRGBvalue
     * @author Sunny
     * @date 24-11-2025
     * @param R Red component
     * @param G Green component
     * @param B Blue component
     * @details Constrains RGB values to 0–255 and returns an RGB struct.
     * @return RGB Struct containing constrained values
    */
    RGB checkRGBvalue(int &R, int &G, int &B) {
        RGB rgb;
        R = constrain(R, 0, MAX_COLOUR_VALUE);
        G = constrain(G, 0, MAX_COLOUR_VALUE);
        B = constrain(B, 0, MAX_COLOUR_VALUE);
        rgb.R = R;
        rgb.G = G;
        rgb.B = B;
        return rgb;
    }
    /**
```

```
     * @name sendBytes
     * @author Sunny
     * @date 26-11-2025
     * @param ptr Pointer to array of bytes to send
     * @param count Number of bytes to send
     * @details Sends bytes to LEDs following the WS2812 protocol using precise timing and
 direct port writes.
    */
    void sendBytes(uint8_t *ptr, int count) {
        uint8_t b, bit, next, hi = bitMask, lo = 0;
        bit = 8;
        b = *ptr++;
        asm volatile(
            "head20%=:\n\t"
            "st %a[port], %[hi]\n\t"
            "sbrc %[byte], 7\n\t"
            "mov %[next], %[hi]\n\t"
            "dec %[bit]\n\t"
            "st %a[port], %[next]\n\t"
            "mov %[next], %[lo]\n\t"
            "breq nextbyte20%=\n\t"
            "rol %[byte]\n\t"
            "rjmp .+0\n\t"
            "nop\n\t"
            "st %a[port], %[lo]\n\t"
            "nop\n\t"
            "rjmp .+0\n\t"
            "rjmp head20%=\n\t"
            "nextbyte20%=:\n\t"
            "ldi %[bit], 8\n\t"
            "ld %[byte], %a[ptr]+\n\t"
            "st %a[port], %[lo]\n\t"
            "nop\n\t"
            "sbiw %[count], 1\n\t"
            "brne head20%=\n\t"
            : [port] "+e"(port), [byte] "+r"(b), [bit] "+r"(bit), [next] "+r"(next), [coun
t] "+w"(count)
            : [ptr] "e"(ptr), [hi] "r"(hi), [lo] "r"(lo)
        );
    }
public:
    /**
     * @name Neopixel
     * @author Sunny
     * @date 26-11-2025
     * @param dataPin Arduino pin connected to the LED strip
     * @param numberOfLeds Total number of LEDs on the strip
     * @details Constructor initializes color arrays and sets all LEDs off
    */
    Neopixel(int dataPin, int numberOfLeds) {
        PIN_NI = dataPin;
        NUMBER_OF_LED = numberOfLeds;
        color = new RGB[NUMBER_OF_LED];
        lastColor = new RGB[NUMBER_OF_LED];
        for (int i = 0; i < NUMBER_OF_LED; i++) {
            color[i] = {0, 0, 0};
            lastColor[i] = {0, 0, 0};
        }
    }
    /**
```

```
     * @name ~Neopixel
     * @author Sunny
     * @date 26-11-2025
     * @details Destructor frees allocated color arrays to prevent memory leaks
    */
    ~Neopixel() {
        delete[] color;
        delete[] lastColor;
    }
    /**
     * @name begin
     * @author Sunny
     * @date 26-11-2025
     * @details Initializes the data pin and determines PORT register and bit mask
    */

    void begin() {
        pinMode(PIN_NI, OUTPUT);
        digitalWrite(PIN_NI, LOW);
        initinfo();
    }
    /**
     * @name fill
     * @author Sunny
     * @date 26-11-2025
     * @param start Start LED index
     * @param end End LED index (-1 = last LED)
     * @param R Red component
     * @param G Green component
     * @param B Blue component
     * @details Sets a range of LEDs to the specified color
    */
    void fill(int start = 0, int end = -1, int R = 0, int G = 0, int B = 0) {

      if (end == -1) {
        end = NUMBER_OF_LED - 1;
      }

      if (start < 0) {
        start = 0;
      }

      if (start > end) {

        start = end;
      }

        RGB rgb = checkRGBvalue(R, G, B);
      for (int i = start; i <= end; i++) {
          color[i] = rgb;
      }
    }
    /**
     * @name clear
     * @author Sunny
     * @date 26-11-2025
     * @details Turns off all LEDs and updates the strip
    */
    void clear() {
        for (int i = 0; i < NUMBER_OF_LED; i++) {
```

```
            color[i] = {0, 0, 0};
        }
        show();
    }
    /**
     * @name show
     * @author Sunny
     * @date 26-11-2025
     * @details Sends current color array to the LED strip using WS2812 protocol
    */
    void show() {
        if (!canShow()) {
          return;
        }

        memcpy(lastColor, color, NUMBER_OF_LED * sizeof(RGB));
        noInterrupts();
        sendBytes((uint8_t *)color, NUMBER_OF_LED * 3);
        delayMicroseconds(300);                                    // Reset signal
        interrupts();
    }
    /**
     * @name setNeoPixelColor
     * @author Sunny
     * @date 26-11-2025
     * @param index LED index
     * @param R Red component
     * @param G Green component
     * @param B Blue component
     * @details Sets a single LED to the specified RGB color
    */
    void setNeoPixelColor(int index, int R, int G, int B) {
        RGB rgb = checkRGBvalue(R, G, B);
        color[index] = rgb;
    }
    /**
     * @name canShow
     * @author Sunny
     * @date 27-11-2025
     * @details Checks if the current color array differs from the last shown array
     * @return bool True if colors have changed, otherwise false
    */
    bool canShow() {
        for (int i = 0; i < NUMBER_OF_LED; i++) {
            if (color[i].R != lastColor[i].R || color[i].G != lastColor[i].G || color[i].B
 != lastColor[i].B) {
                return true;
            }
        }
        return false;
    }
};
```

/tests/

/test_reflective_sensor/

(/common/tools/tests/test_reflective_sensor.cpp)

```
#include "test_reflective_sensor.h"
/**
 * @name testReflectiveSensor
 * @author Fumbre (Vladyslav)
 * @date 21-11-2025
 * @details make test for reflective sensor
*/
void testReflectiveSensor() {
    Serial.println(analogRead(A0));
    Serial.println(analogRead(A1));
    Serial.println(analogRead(A2));
    Serial.println(analogRead(A3));
    Serial.println(analogRead(A4));
    Serial.println(analogRead(A5));
    Serial.println(analogRead(A6));
    Serial.println(analogRead(A7));
}
```

/test_sonar/

(/common/tools/tests/test_sonar.cpp)

```
#include "test_sonar.h"
/**
 * @name testSonar
 * @author Fumbre (Vladyslav)
 * @date 21-11-2025
 * @details make test for sonar
*/
void testSonar() {
    Serial.println(getDistanceCM_Front());
}
/**
 * @name testSonars
 * @author Uraib
 * @date 19-12-2025
 * @details make 3 sonars tests
*/
void testSonars() {
    static Timer t;
    if (!t.intervalStart(400)) {
        return;
    }

    float f = getDistanceCM_Front();
    float l = getDistanceCM_Left();
    float r = getDistanceCM_Right();
    Serial.print("STATE | ");
    // front
    Serial.print("F:");
    if (f > 1.0) {
        Serial.print(f, 2);
    } else {
        Serial.print("X");
```

```
    }

    // left
    Serial.print("  L:");
    if (l > 1.0) {
        Serial.print(l, 2);
    } else {
        Serial.print("X");
    }

    // right
    Serial.print("  R:");
    if (r > 1.0) {
        Serial.print(r, 2);
    } else {
        Serial.print("X");
    }

    // visual flags
    Serial.print(" | [");
    // Front obstacle
    if (f > 1.0 && f < 18.0) {
        Serial.print("F");
    } else {
        Serial.print(".");
    }

    // Left obstacle
    if (l > 1.0 && l < 15.0) {
        Serial.print("L");
    } else {
        Serial.print(".");
    }

    // Right obstacle
    if (r > 1.0 && r < 15.0) {
        Serial.print("R");
    } else {
        Serial.print(".");
    }
    Serial.println("]");
}
```

/test_pulses/

(/common/tools/tests/test_pulses.cpp)

```
#include "test_pulses.h"
Timer pulseStamp;
/**
 * @name testPulses
 * @author Fumbre (Vladyslav)
 * @date 21-11-2025
 * @details make test for pulses
*/
void testPulses(int updateTime) {
  if (pulseStamp.intervalStart(updateTime)) {
    Serial.print("Left pulses: ");
    Serial.print(motor_left_pulses_counter);
```

```
    Serial.print(" Right pulses: ");
    Serial.println(motor_right_pulses_counter);
  }
}
```

Behaviour Programs

/maps_program/follow_single_line/

```
/**
 * @name follow_single_line
 * @authors Fumbre (Vladyslav) & Aria & Sunny
 * @date 15-12-2025
 * @details Controls the robot to follow a single line maze using reflective sensors and s
onar.
 * @details Includes obstacle avoidance, maze sequence handling, LED feedback, and motor c
ontrol.
*/
#include "follow_single_line.h"
ReflectiveSensor rsLine(PINS_RS, PINS_RS_LENGTH, 220, 35);
Sequence mazeSequence(&rsLine);
// Maze state variables
bool mazePassed = false;
bool isEndSequence = false;
bool isMazeStarted = false;
bool safeZone = true;
// Movement configuration
int fullSpeed = 255;
float slightConf = 0.8;          // speed multiplier for slight turns
float hardConf = -0.5;           // speed multiplier for hard turns
static Timer t;
/**
 * @name followLine
 * @authors Fumbre (Vladyslav) & Aria & Sunny
 * @date 15-12-2025
 * @details Main loop for following a single line in a maze.
 * @details Handles maze start, reflective sensor reading, obstacle avoidance, motor contr
ol, and maze completion.
*/
void followLine() {
    if (!isMazeStarted) {
        if (mazeSequence.readyToStart(1)) {
            isMazeStarted = true;
        }
        return;
    }
    if (t.executeOnce(0)) {
        moveSpeed(230, 230);
    }
    LineState currnetPattern = rsLine.pattern();
    if (!isEndSequence) {
        dataSend();
        if (!mazeSequence.start(255, 8, 180))
            return;
    }
    if (!isEndSequence) {
        if (!avoiding) {
```

```
            if (t.interval(35, 75)) {
                float distance = getDistanceCM_Front();
                if (safeZone && distance <= 13 && distance >= 2) {
                    safeZone = false;
                    obstacleAvoidance(255);
                    return;
                }
                if (distance > 13) {
                    safeZone = true;
                }
            }
        } else {
            obstacleAvoidance(255);
            return;
        }
        solvingFollowSingleLine(currnetPattern, fullSpeed, slightConf, hardConf);
    }
    if (isEndSequence) {
        if (!t.timeout(1000)) {
            dataSend();
        }
        mazeSequence.end(&mazePassed, "BB016");
    }
}
/**
 * @name solvingFollowSingleLine
 * @authors Fumbre (Vladyslav) & Aria & Sunny
 * @date 15-12-2025
 * @param currnetPattern Current reflective sensor pattern
 * @param fullSpeed Base motor speed
 * @param slightConf Multiplier for slight turns
 * @param hardConf Multiplier for hard turns
 * @details Implements the core logic for following the line based on the current sensor p
attern.
 * @details Controls motor speed adjustments and LED feedback according to pattern.
*/
void solvingFollowSingleLine(LineState currnetPattern, int fullSpeed, float slightConf, fl
oat hardConf) {
    switch (currnetPattern) {
    case CENTER:
        moveSpeed(fullSpeed, fullSpeed);
        turnOnAllLeds(0, 255, 0);
        break;
    case SLIGHT_LEFT:
        moveSpeed(fullSpeed * slightConf, fullSpeed);
        break;
    case SLIGHT_RIGHT:
        moveSpeed(fullSpeed, fullSpeed * slightConf);
        break;
    case HARD_LEFT:
        moveSpeed(fullSpeed * hardConf, fullSpeed);
        turnOnSomeLeds((int[]){3, 0}, 2, 255, 255, 0);
        turnOnSomeLeds((int[]){1, 2}, 2, 0, 255, 0);
        break;
    case HARD_RIGHT:
        moveSpeed(fullSpeed, fullSpeed * hardConf);
        turnOnSomeLeds((int[]){1, 2}, 2, 255, 255, 0);
        turnOnSomeLeds((int[]){3, 0}, 2, 0, 255, 0);
        break;
    case ALL_BLACK:
```

```
        isEndSequence = mazeSequence.isDetecetingBlackSquare(80);
        moveSpeed(fullSpeed, fullSpeed);
        break;
    }
}
/**
 * @name followLineSetup
 * @authors Fumbre (Vladyslav)
 * @date 15-12-2025
 * @details Initializes all components required for following a line maze.
 * @details Sets up HC12 communication, motors, gripper, sonar, and reflective sensors.
*/
void followLineSetup() {
    buildHC12Connection();
    setupMotor();
    setupGripper();
    gripperUnCatch();
    setupSonar();
    rsLine.setup();
}
```

/maps_program/maze_line/

```
#include "maze_line.h"
ReflectiveSensor rsLine2(PINS_RS, PINS_RS_LENGTH, 200, 25);
Sequence mazeSequence2(&rsLine2);
LineState lastStatus = CENTER;
LineState currentStatus;
// Speed configuration
int baseSpeed2 = 230;
float slightConf2 = 0.7;
float hardConf2 = 0;
float reverseConf2 = -1;
bool doRotationRight = false;
bool doRotationLeft = false;
bool rotated = false;
// Maze variables
bool isMazeStarted2 = false;
bool isEndSequence2 = false;
bool mazePassed2 = false;
bool startOnce = false;
static Timer t;
static Timer t1;
void mazeLine() {
  // set poisition of robot
  if (!isMazeStarted2) {
    if (mazeSequence2.readyToStart(2)) {
      isMazeStarted2 = true;
      startOnce = true;
    }
    return;
  }
  if (startOnce) {
    moveSpeed(baseSpeed2, baseSpeed2);
    startOnce = false;
  }
  if (!isEndSequence2) {
    dataSend();
```

```
    if (!mazeSequence2.start(220, 10))
      return;
  }
  // If rotation do only rotation
  if (doRotationRight) {
    rotate(0);
    return;
  }
  // Left one
  if (doRotationLeft) {
    rotate(1);
    return;
  }
  if (!isEndSequence2) {
    if (t.interval(35, 70)) {
      float distance = getDistanceCM_Front();
      if (distance < 18) {
        doRotationRight = true;
        rotate(0);
        return;
      }
    }
    currentStatus = rsLine2.pattern();
    // Main maze line code
    switch (currentStatus) {
    case CENTER:
      if (lastStatus == ALL_BLACK) {
        doRotationRight = true;
        rotate(0);
        break;
      }
      lastStatus = CENTER;
      moveSpeed(baseSpeed2, baseSpeed2);
      break;
    case SLIGHT_LEFT:
      if (lastStatus == ALL_BLACK) {
        doRotationRight = true;
        rotate(0);
        return;
      }
      lastStatus = CENTER;
      moveSpeed(baseSpeed2 * slightConf2, baseSpeed2);
      break;
    case SLIGHT_RIGHT:
      if (lastStatus == ALL_BLACK) {
        doRotationRight = true;
        rotate(0);
        return;
      }
      lastStatus = CENTER;
      moveSpeed(baseSpeed2, baseSpeed2 * slightConf2);
      break;
    case ALL_BLACK:
      isEndSequence2 = mazeSequence2.isDetecetingBlackSquare(150);
      lastStatus = ALL_BLACK;
      moveSpeed(baseSpeed2 * .8, baseSpeed2 * .9);
      break;
    case ALL_WHITE:
      if (lastStatus == ALL_BLACK || lastStatus == RIGHT_TURN || lastStatus == CENTER) {
        doRotationRight = true;
```

```
        rotate(0);
      }
      if (lastStatus == LEFT_TURN)
      {
        doRotationLeft = true;
        rotate(1);
      }
      break;
    case HARD_LEFT:
      moveSpeed(baseSpeed2 * hardConf2, baseSpeed2 * slightConf2);
      break;
    case LEFT_TURN:
      lastStatus = LEFT_TURN;
      moveSpeed(baseSpeed2, baseSpeed2);
      break;
    case HARD_RIGHT:
      moveSpeed(baseSpeed2 * slightConf2, baseSpeed2 * hardConf2);
      break;
    case RIGHT_TURN:
      lastStatus = RIGHT_TURN;
      doRotationRight = true;
      delay(15);
      rotate(0);
      break;
    }
  }
  // Send start command to BB011
  if (isEndSequence2) {
    if (!t1.timeout(1000)) {
      dataSend();
    }
    mazeSequence2.end(&mazePassed2, "BB046");
  }
}
// dir == 0 -right; dir == 1 -left
void rotate(int dir) {
  bool end = false;
  if (!t.timeout(200)) {                    // execute for 200 milliseconds
    moveSpeed(baseSpeed2, baseSpeed2);
  }
  else {
    if (dir == 0) {
      if (!rotated) {
        if (t.executeOnce(0, 300)) {
          moveSpeed(200, -200);
          // return;
        }
        else {
          rotated = true;
        }
      }
    }
    if (dir == 1) {
      if (!rotated) {
        if (t.executeOnce(0, 300)) {
          moveSpeed(-200, 200);
        }
        else {
          rotated = true;
        }
```

```
      }
    }
    if (rotated) {
      LineState pattern = rsLine2.pattern();
      if (pattern == CENTER || pattern == SLIGHT_LEFT || pattern == SLIGHT_RIGHT) {
        moveStopAll();
        end = true;
        lastStatus = CENTER;
        moveSpeed(170, 170);
        turnOnAllLeds(0, 255, 0);
      }
    }
  }
  if (end) {
    rotated = false;
    doRotationRight = false;
    doRotationLeft = false;
    t.resetTimeout();
    t.resetExecuteOnce();
  }
}
void mazeLineSetup() {
  buildHC12Connection();
  setupMotor();
  setupGripper();
  setupSonar();
  rsLine2.setup();
  gripperUnCatch();
}
```

/maps_program/physical_maze/

```
/**
 * @name physical maze - PRO VERSION
 * @author Francisco & Uraib
 * @description Timed movement with Active Safety and Right-Hand Rule Priority.
*/
#include "physical_maze.h"
const int FWD_SPEED = 230;
const int TURN_SPEED = 180;
const int TURN_90_MS = 520;
const int TURN_180_MS = 850;
const int FORWARD_TIME_MS = 600;
const int REVERSE_TIME_MS = 600; // Increased to ensure it clears the wall
const int CHECK_PAUSE_MS = 250;
const float WALL_DIST = 16.0;
const float EMERGENCY_STOP_DIST = 10.0;
const float MAX_DIF = 2.0;
float lastF = -1;
float lastR = -1;
float lastL = -1;
enum MazeState {
    MAZE_FORWARD_TIMED,
    MAZE_CHECK,
    MAZE_TURN_LEFT_90,
    MAZE_TURN_RIGHT_90,
    MAZE_TURN_180,
    MAZE_REVERSE
```

```
};
int mazeState = MAZE_FORWARD_TIMED;
static Timer actionTimer;
// The "Snapshots"
float distBeforeMove = -1;
// RS - reflective sensor
ReflectiveSensor rsLine3(PINS_RS, PINS_RS_LENGTH, 200, 25);
Sequence mazeSequence3(&rsLine3);
// maze variables
bool isMazeStarted3 = false;
bool isEndSequence3 = false;
bool mazePassed3 = false;
bool startOnce2 = true;
bool turn180Left = true;
void physicalMazeSetup() {
    buildHC12Connection();
    setupSonar();
    setupMotor();
    setupGripper();
}
void physicalMaze() {
    static Timer t;
    // set poisition of robot
    if (!isMazeStarted3) {
        if (mazeSequence3.readyToStart(1)) {
            isMazeStarted3 = true;
        }
        return;
    }
    if (startOnce2) {
        moveSpeed(FWD_SPEED, FWD_SPEED);
        startOnce2 = false;
    }
    if (!isEndSequence3) {
        if (!mazeSequence3.start(230, 9, 280))
            return;
    }
    if (!isEndSequence3) {
        LineState currentStatus = rsLine3.pattern();
        switch (currentStatus) {
        case CENTER:
            moveSpeed(FWD_SPEED, FWD_SPEED);
            break;
        case SLIGHT_LEFT:
            moveSpeed(FWD_SPEED * .8, FWD_SPEED);
            break;
        case SLIGHT_RIGHT:
            moveSpeed(FWD_SPEED, FWD_SPEED * .8);
            break;
        case HARD_LEFT: {
            moveSpeed(FWD_SPEED * 0, FWD_SPEED);
            break;
        };
        case HARD_RIGHT: {
            moveSpeed(FWD_SPEED, FWD_SPEED * 0);
            break;
        };
        case ALL_WHITE:
            solvingPhysicalMaze();
            break;
```

```
        case ALL_BLACK:
            isEndSequence3 = mazeSequence3.isDetecetingBlackSquare(80);
            break;
        }
    }
    if (isEndSequence3) {
        mazeSequence3.end(&mazePassed3, "BB011");
    }
}
void solvingPhysicalMaze() {
    switch (mazeState) {
    case MAZE_FORWARD_TIMED:
        // Snapshot 1: Right when the state starts
        if (distBeforeMove < 0) {
            distBeforeMove = getDistanceCM_Front();
            actionTimer.resetTimeout();
        }
        if (!actionTimer.timeout(FORWARD_TIME_MS)) {
            float currentF = getDistanceCM_Front();
            // Emergency stop if we get too close
            if (currentF < EMERGENCY_STOP_DIST) {
                moveStopAll();
                distBeforeMove = -1;
                mazeState = MAZE_CHECK;
            } else {
                moveStabilized(FWD_SPEED, FWD_SPEED);
            }
        }
        else {
            moveStopAll();
            float distAfterMove = getDistanceCM_Front();
            if (abs(distBeforeMove - distAfterMove) < 4.0 && distAfterMove < 30.0) {
                mazeState = MAZE_REVERSE;
            }
            else {
                mazeState = MAZE_CHECK;
            }
            distBeforeMove = -1;
            actionTimer.resetTimeout();
        }
        break;
    case MAZE_CHECK: {
        moveStopAll();
        delay(CHECK_PAUSE_MS);
        float r = getDistanceCM_Right();
        float f = getDistanceCM_Front();
        float l = getDistanceCM_Left();
        if (lastF < 0) {
            lastF = f;
            lastR = r;
            lastL = l;
        }
        // Only reverse if passed grace period
        if (abs(f - lastF) < MAX_DIF && abs(l - lastL) < MAX_DIF && abs(r - lastR) < MAX_D
IF) {
            actionTimer.resetTimeout();
            mazeState = MAZE_REVERSE;
        }
        else {
            // right free
```

```
            if (r > WALL_DIST) {
                mazeState = MAZE_TURN_RIGHT_90;
            }
            else if (f > WALL_DIST) { // front free
                mazeState = MAZE_FORWARD_TIMED;
            }
            else if (l > WALL_DIST) { // left free
                mazeState = MAZE_TURN_LEFT_90;
            }
            else {
                mazeState = MAZE_TURN_180; // 3 walls
            }
        }
        // Update last values
        lastF = f;
        lastL = l;
        lastR = r;
        actionTimer.resetTimeout();
        break;
    }
    case MAZE_REVERSE:
        if (!actionTimer.timeout(REVERSE_TIME_MS)) {
            moveStabilized(-FWD_SPEED, -FWD_SPEED);
        }
        else {
            moveStopAll();
            actionTimer.resetTimeout();
            // Force a turn so it doesn't just hit the same wall again
            mazeState = MAZE_TURN_LEFT_90;
        }
        break;
    case MAZE_TURN_LEFT_90:
        if (!actionTimer.timeout(TURN_90_MS)) {
            moveStabilized(-TURN_SPEED, TURN_SPEED);
        }
        else {
            moveStopAll();
            delay(100);
            actionTimer.resetTimeout();
            mazeState = MAZE_FORWARD_TIMED;
        }
        break;
    case MAZE_TURN_RIGHT_90:
        if (!actionTimer.timeout(TURN_90_MS)) {
             moveStabilized(TURN_SPEED, -TURN_SPEED);
        }
        else {
            moveStopAll();
            delay(100);
            actionTimer.resetTimeout();
            mazeState = MAZE_FORWARD_TIMED;
        }
        break;
    case MAZE_TURN_180:
        if (!actionTimer.timeout(TURN_180_MS)) {
            moveStabilized(TURN_SPEED, -TURN_SPEED);
        }
        else {
            moveStopAll();
            actionTimer.resetTimeout();
```

```
            mazeState = MAZE_CHECK;
        }
        break;
    }
}
```

Debriefing

We are creating maze solutions that can work on different robot hardware. Depending on the maze settings, all robots should be able to pass it. The only limitation is the physical maze, because it uses three sonar sensors. That is why only robots equipped with three sonars can solve this maze.

While solving the maze, each robot sends data to Arduino. Arduino uses a proxy (Node.js) to connect to the real world and sends this data via WebSocket in real time. The web page receives this data and displays it immediately.

In the end, we have:

- a first project with multiple maze solutions for robots,
- and a second project that shows real-time data from active robots.

Uraib & Francisco – physical maze

Aria & Sunny & Vladyslav – line mazes

Sunny & Vladyslav - WEB

The deadline is January 14.

Additional Links

Git robot - https://github.com/Fumbre/Retro2Cbot

Git web - https://github.com/Fumbre/Retro2Cbot_web

YouTube Video - https://www.youtube.com/watch?v=ckTkvnyR-1Q

Web stie - https://retro2cbot-web.onrender.com

Backend - https://retro2cbot-web-dugz.onrender.com